Rockchip HDMI RX Developer Guide

ID: RK-KF-YF-321

Release Version: V1.1.7

Release Date: 2024-04-28

Security Level: □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip's registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2024. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

This document is a guide for developing the HDMI IN functionality using the HDMI RX module based on the RK3588 Android 12 platform.

Overview

The HDMI IN functionality can be implemented via bridge chips, converting HDMI signals to MIPI signals for reception. The RK3588 chip platform features a built-in HDMI RX module capable of directly receiving HDMI signals. This document primarily introduces the method for developing the HDMI IN function via the HDMI RX module on the RK3588 Android 12 platform. It supports adaptive HDMI IN resolutions: 4K60, 4K30, 1080P60, 720P60, etc. It also supports HDMI IN hot-plugging, recording functionality, configurable EDID, HDCP1.4/HDCP2.3, and CEC.

Product Version

Chip Name	Kernel Version	Android Version
RK3588	Linux 5.10	Android12

Intended Audience

This document (this guide) is mainly intended for:

Technical Support Engineer

Software Development Engineer

Revision History

Version	Author	Modification Date	Modification Description
V1.0.0	Wen Dingxian/Guo Wangqiang/Wang Hang/Chen Shunqing/Lan Shunhua	2022.06.29	Initial version
V1.1.0	Chen Shunqing	2022.08.09	Modified HDCP and CEC instructions
V1.1.1	Wang Hang	2022.11.04	Modified HDMIIN APK instructions
V1.1.2	Chen Shunqing	2023.4.12	Updated configuration instructions for HDCP/CEC
V1.1.3	Wang Hang	2023.11.17	Added configuration instructions for vtunnel and MIPI of the HDMIIN APK
V1.1.4	Guo Wangqiang	2023.11.20	Added troubleshooting methods for common issues and log descriptions
V1.1.5	Wang Hang	2024.01.30	Updated instructions regarding reserved CMA memory during TIF preview

Version	Author	Modification Date	Modification Description
V1.1.6	Wang Hang	2024.03.04	Updated node attribute configuration for RK3576 MIPI preview
V1.1.7	Guo Wangqiang	2024.04.28	Added typical logs and patch notes for reboot issues

Contents

Rockchip HDMI RX Developer Guide

1. HDMI IN Function Overview
    - 1.1 Introduction to HDMI RX Module Features
    - 1.2 HDMI IN Functional Block Diagram
2. HDMI RX Driver Code and dts Configuration
    - 2.1 SDK Version Requirements
    - 2.2 Driver Code and Kernel Configuration
    - 2.3 dts Configuration Description
        - 2.3.1 HDMI RX Controller Configuration
        - 2.3.2 Reserved Memory
        - 2.3.3 Audio Configuration
        - 2.3.4 HDCP Configuration
        - 2.3.5 CEC Configuration
    - 2.4 EDID Configuration Method
3. HDMI IN Video Architecture
    - 3.1 HDMI IN Video Workflow
    - 3.2 Main Driver Architecture of HDMI RX
    - 3.3 Image Buffer Rotation Mechanism
    - 3.4 Image Transmission Latency
4. HDMI IN HDCP Function
    - 4.1 HDCP1.4
        - 4.1.1 dts Configuration
        - 4.1.2 Key Flashing
        - 4.1.3 HDCP1.4 Status View
    - 4.2 HDCP2.3
        - 4.2.1 dts Configuration
        - 4.2.2 Packaging Firmware and Boot Services
        - 4.2.3 HDCP2.3 Status View
    - 4.3 HDCP KEY Flashing
5. HDMI IN CEC Function
6. HDMI IN APK Adaptation Method
    - 6.1 APK Source Code Path
    - 6.2 HdmiIn Preview APK Instructions
    - 6.3 Differences Between TIF and Camera Preview Modes
7. Driver Debugging Methods
    - 7.1 Debugging Tool Acquisition
    - 7.2 Debugging Command Examples
        - 7.2.1 View the video node of HDMIRX
        - 7.2.2 Locate the rk_hdmirx Device
        - 7.2.3 Retrieve Driver Timing Information
        - 7.2.4 Real-time Query of Timing Information
        - 7.2.5 Query Resolution and Image Format
        - 7.2.6 Enable Image Data Stream
        - 7.2.7 Grabbing Image Files
        - 7.2.8 Normal Stream Retrieval Log
8. Common Troubleshooting Methods
    - 8.1 Enable the Log Switch
    - 8.2 Read and Write Registers via io Commands
    - 8.3 HDMI RX Status Query
    - 8.4 HDMI IN Signal Lock Failure
    - 8.5 HDMI IN No Output/Black Screen Issue
    - 8.6 Signal Instability and Loss of Lock Issue
    - 8.7 Source Resolution Switching Failed
    - 8.8 Driver Statistics TMDSCLK Errors
    - 8.9 Source-End Special Scenario: Brief Black Screen

9. Typical Log Description
    9.1 Hot-Swap Logs
    9.2 Resolution Switching Log
    9.3 Driver Switch Flow Log
    9.4 HDMI IN Data Stream Printing
    9.5 Signal Unlocked Exception Log
    9.6 Loss of Lock After Signal Acquisition
    9.7 TMDSCLK Statistical Errors
    9.8 Hot-plugging or Resolution Change Reboot
10. Patch Notes Regarding Restart Issues

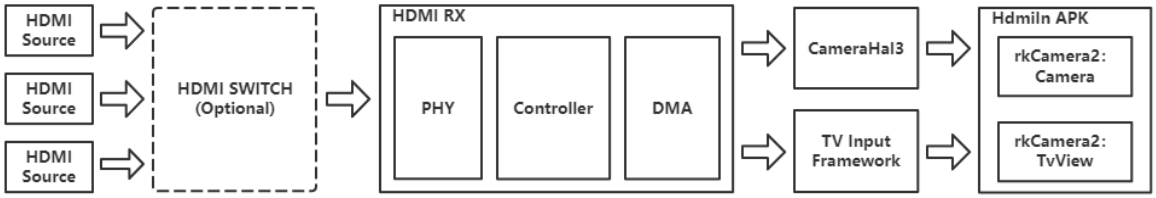
1. HDMI IN Function Overview

1.1 Introduction to HDMI RX Module Features

- HDMI 1.4b/2.0 RX: Up to 4K@60fps
- Support FMT: RGB888/YUV420/YUV422/YUV444 8bit
- Pixel clock: Up to 600MHz
- HDCP1.4/2.3
- CEC hardware engine
- E-EDID configuration
- S/PDIF 2channel output
- I2S 2/4/6/8channel output

Note: RK3588S does not include the HDMI RX module.

1.2 HDMI IN Functional Block Diagram



Based on application scenario requirements, the HDMI RX can be adapted to either the TIF framework or the Camera framework. Adapting to the TIF framework results in lower image transmission latency, while adapting to the Camera framework allows the use of the standard Camera API, facilitating application function development such as video recording, integration with backend algorithms, and other features.

2. HDMI RX Driver Code and dts Configuration

2.1 SDK Version Requirements

The HDMI IN feature may undergo continuous updates. It is recommended to upgrade the SDK version to the latest. The method to query the SDK version number is as follows:

```
rk3588_s:/ # getprop | grep rksdk
[ro.rksdk.version]: [ANDROID12_RKR9]
```

2.2 Driver Code and Kernel Configuration

Driver Code:

```
drivers/media/platform/rockchip/hdmirx/
```

Kernel Configuration:

```
CONFIG_VIDEO_ROCKCHIP_HDMIRX=y
```

2.3 dts Configuration Description

Refer to the dts configuration of RK3588 EVB1 in the SDK:

```
arch/arm64/boot/dts/rockchip/rk3588-evb1-lp4.dtsi
```

2.3.1 HDMI RX Controller Configuration

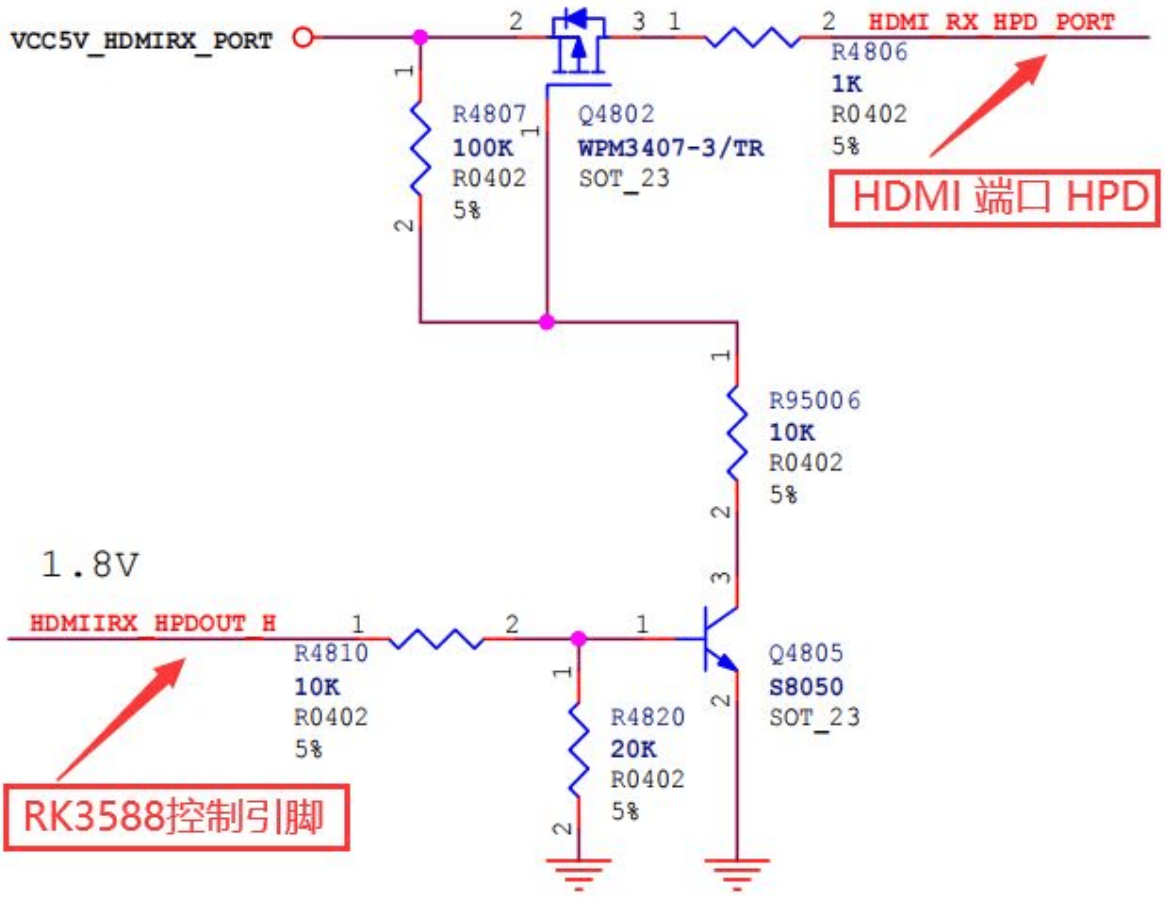
```
/* Should work with at least 128MB cma reserved above. */
&hdmirx_ctrler {
    status = "okay";

    /* Effective level used to trigger HPD: 0-low, 1-high */
    hpd-trigger-level = <1>;
    hdmirx-det-gpios = <&gpio2 RK_PB5 GPIO_ACTIVE_LOW>;
    pinctrl-names = "default";
    pinctrl-0 = <&hdmim1_rx &hdmirx_det>;
};

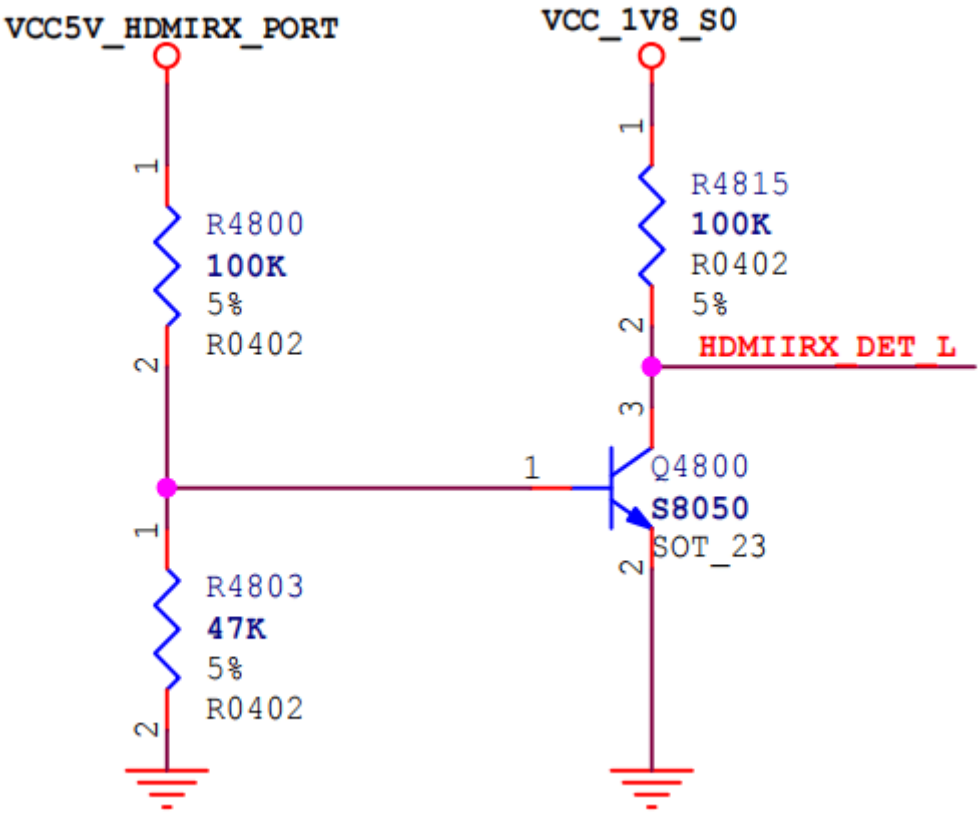
hdmi {
    hdmirx_det: hdmirx-det {
        rockchip,pins = <2 RK_PB5 RK_FUNC_GPIO &pcfg_pull_up>;
    };
};
```

Board-level configuration must correspond to the actual hardware circuit connections:

- hpd-trigger-level: The active level for triggering HPD. <1> indicates that the HPD level status of the RK3588 control pin is the same as that of the HDMI port, while <0> indicates the opposite.



- hdmirx-det-gpios: HDMI insertion detection pin. Configure the GPIO and active level according to the actual hardware connection. For active-low configuration, set the pinctrl to internal pull-up.



2.3.2 Reserved Memory

The RK3588 HDMI RX module can only use physically contiguous memory and requires reserving at least 128MB of CMA memory. If an increase is needed, simply modify the reserved memory length in the dts.

Note: Calculated based on 3840x2160 resolution, RGB888 image format, and 4 rotating buffers.

```
/* If hdmirx node is disabled, delete the reserved-memory node here. */
reserved-memory {
    #address-cells = <2>;
    #size-cells = <2>;
    ranges;

    /* Reserve 128MB memory for hdmirx-controller@fdee0000 */
    cma {
        compatible = "shared-dma-pool";
        reusable;
        /* Start address: 256 * 0x100000, Length: 128 * 0x100000 */
        reg = <0x0 (256 * 0x100000) 0x0 (128 * 0x100000)>;
        linux,cma-default;
    };
};
```

2.3.3 Audio Configuration

Add sound card configuration in the DTS

```
diff --git a/arch/arm64/boot/dts/rockchip/rk3588-evb1-lp4.dtsi
b/arch/arm64/boot/dts/rockchip/rk3588-evb1-lp4.dtsi
index 4aa6a8ce9364..84c9c51d3b7f 100644
--- a/arch/arm64/boot/dts/rockchip/rk3588-evb1-lp4.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3588-evb1-lp4.dtsi
@@ -47,6 +47,26 @@ play-pause-key {
                };
        };
+       hdmiin_dc: hdmiin-dc {
+               compatible = "rockchip,dummy-codec";
+               #sound-dai-cells = <0>;
+       };
+
+       hdmiin-sound {
+               compatible = "simple-audio-card";
+               simple-audio-card,format = "i2s";
+               simple-audio-card,name = "rockchip,hdmiin";
+               simple-audio-card,bitclock-master = <&dailink0_master>;
+               simple-audio-card,frame-master = <&dailink0_master>;
+               status = "okay";
+               simple-audio-card,cpu {
+                       sound-dai = <&i2s7_8ch>;
+               };
+               dailink0_master: simple-audio-card,codec {
+                       sound-dai = <&hdmiin_dc>;
+               };
+       };
+
        pcie20_avdd0v85: pcie20-avdd0v85 {
                compatible = "regulator-fixed";
```

```
                regulator-name = "pcie20_avdd0v85";
@@ -460,6 +480,10 @@ &i2s6_8ch {
        status = "okay";
 };
+&i2s7_8ch {
+       status = "okay";
+};
+
```

Add HDMI-IN sound card information in the HAL

```
diff --git a/tinyalsa_hal/audio_hw.c b/tinyalsa_hal/audio_hw.c
index ea97bee..509d140 100644
--- a/tinyalsa_hal/audio_hw.c
+++ b/tinyalsa_hal/audio_hw.c
@@ -362,6 +362,7 @@ struct dev_proc_info HDMI_IN_NAME[] =
 {
     {"realtekrt5651co", "tc358749x-audio"},
+    {"rockchiphdmiin", NULL},
     {NULL, NULL}, /* Note! Must end with NULL, else will cause crash */
 };
```

The above configuration is included in the default SDK. If the customer modifies the sound card name, the corresponding sound card information must be added to the HDMI_IN_NAME array in the HAL.

2.3.4 HDCP Configuration

The RK3588 has two HDCP2.3 controllers (hdcp0 and hdcp1), and each HDCP2.3 controller has 3 ports:

hdcp0: DPTX0 and DPTX1

hdcp1: HDMIRX, HDMITX0, and HDMITX1

If HDMIRX needs to support HDCP2.3, hdcp1 must be enabled.

- Support for HDCP1.4 only:

```
&hdmirx_ctrler {
    status = "okay";
    hdcp1x-enable;
};
```

- Supports HDCP2.3. When HDCP2.3 is enabled, it defaults to being compatible with HDCP1.4:

```
&hdcp1 {
    status = "okay";
};

&hdmirx_ctrler {
    status = "okay";
    hdcp2x-enable;
};
```

2.3.5 CEC Configuration

```
&hdmirx_ctrler {
    status = "okay";
    cec-enable;
};
```

2.4 EDID Configuration Method

The RK3588 supports configurable EDID. Currently, the resolutions supported by EDID in the driver code include:

```
3840x2160P60, 3840x2160P50, 3840x2160P30, 3840x2160P25, 3840x2160P24,
1920x1080P60, 1920x1080P50, 1920x1080P30, 1920x1080i60, 1920x1080i50,
1600x900P60, 1440x900P60, 1280x800P60,
1280x720P60, 1280x720P50, 1024x768P60,
720x576P50, 720x480P60, 720x576i50, 720x480i60,
800x600P60, 640x480P60
```

Supported input formats include:

```
RGB888, YUV420, YUV422, YUV444
```

Currently, EDID is divided into two groups, which can be configured via the upper-layer application.

- edid_init_data_340M: Resolutions with a pixel clock less than 340 MHz, primarily supporting HDMI 1.4 specifications, including 4K60 YUV420.
- edid_init_data_600M: Supports resolutions with a pixel clock of 594 MHz, including 4K60 YUV422/YUV444/RGB888.

If EDID configuration is required, the corresponding array can be directly modified in the driver code:

drivers/media/platform/rockchip/hdmirx/rk_hdmirx.c

```
static u8 edid_init_data_340M[] = {
    0x00, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0x00,
    0x49, 0x70, 0x88, 0x35, 0x01, 0x00, 0x00, 0x00,
    0x2D, 0x1F, 0x01, 0x03, 0x80, 0x78, 0x44, 0x78,
    ...
};

static u8 edid_init_data_600M[] = {
    0x00, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0x00,
    0x49, 0x70, 0x88, 0x35, 0x01, 0x00, 0x00, 0x00,
    0x2D, 0x1F, 0x01, 0x03, 0x80, 0x78, 0x44, 0x78,
    ...
};
```

Alternatively, dynamically configure the EDID by invoking the ioctl interface of the video node:

```
static const struct v4l2_ioctl_ops hdmirx_v4l2_ioctl_ops = {
    ...
    .vidioc_g_edid = hdmirx_get_edid,
    .vidioc_s_edid = hdmirx_set_edid,
    ...
};
```

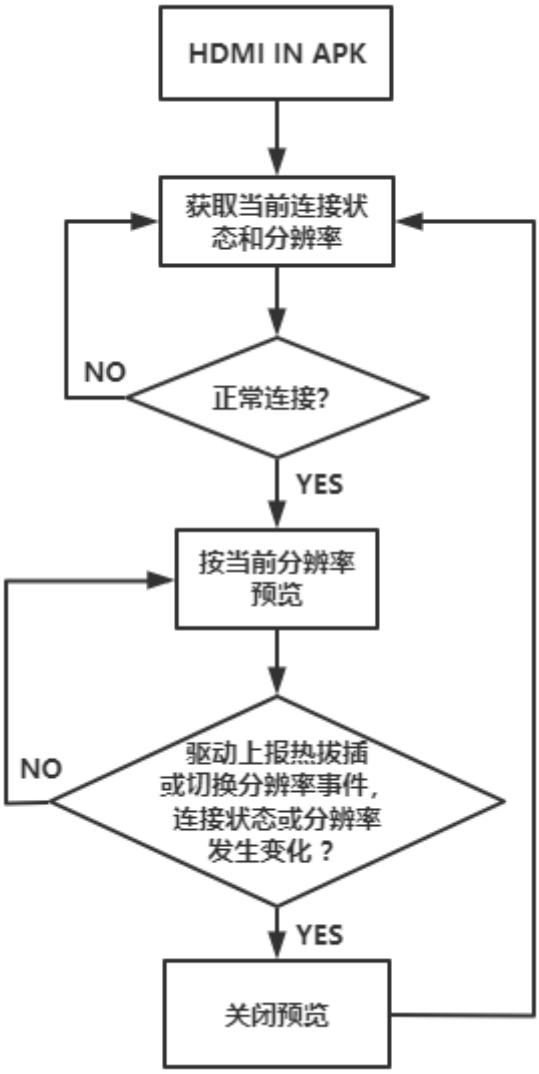
The EDID configuration can be edited using a tool and subsequently integrated into the driver code. EDID visual editing tools can be searched for and downloaded online.

The recommended tools are as follows:

https://980-manager.software.informer.com/download/#google_vignette

3. HDMI IN Video Architecture

3.1 HDMI IN Video Workflow

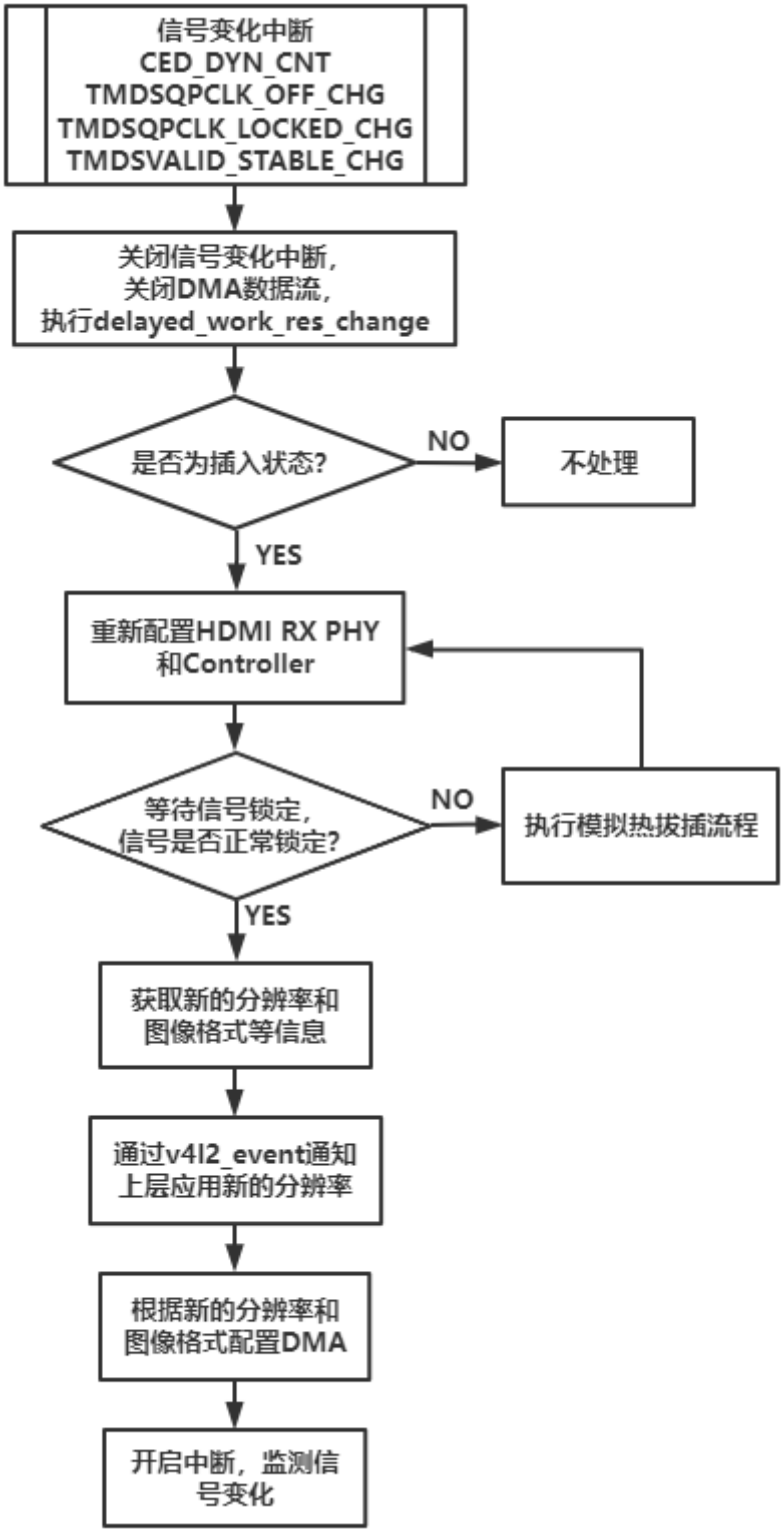


3.2 Main Driver Architecture of HDMI RX

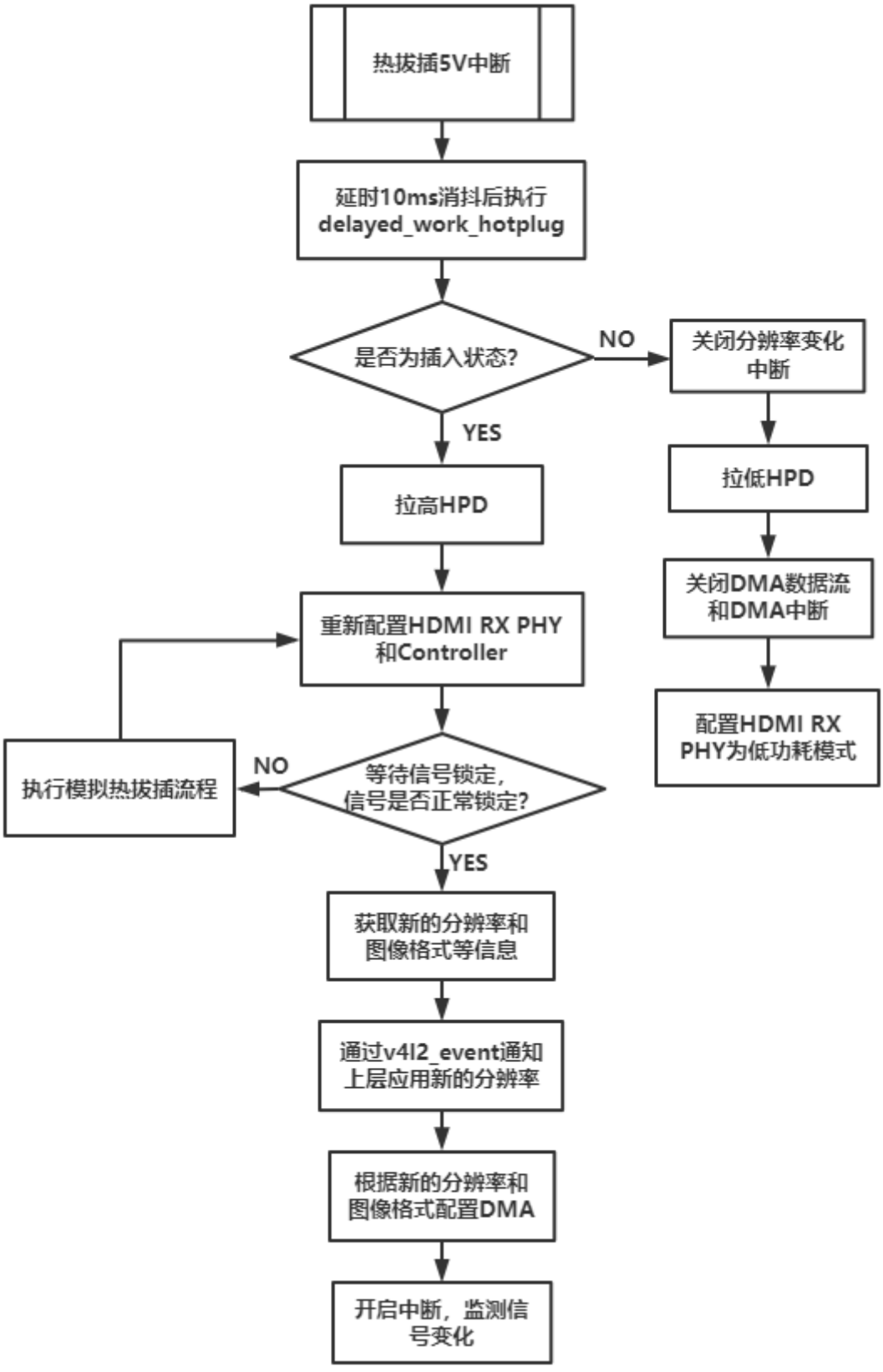
In the HDMI RX driver architecture, the following interrupts and delayed_work items require particular attention:

- hdmirx_5v_det_irq_handler: 5V detection interrupt, triggered by the GPIO pin HDMIIRX_DET_L, used to detect HDMI connection and disconnection events.
- hdmirx_hdmi_irq_handler: HDMI RX controller interrupt, primarily used for initialization configuration and monitoring HDMI signal changes.
- hdmirx_dma_irq_handler: HDMI RX DMA interrupt, primarily utilized during buffer rotation in the image preview process.
- hdmirx_delayed_work_hotplug: The delayed_work corresponding to the 5V detection interrupt. It performs configuration processing on the HDMI RX module when a hot-plug event occurs.
- hdmirx_delayed_work_res_change: Reconfigures the HDMI RX controller and waits for signal lock when the controller detects a signal change interrupt.

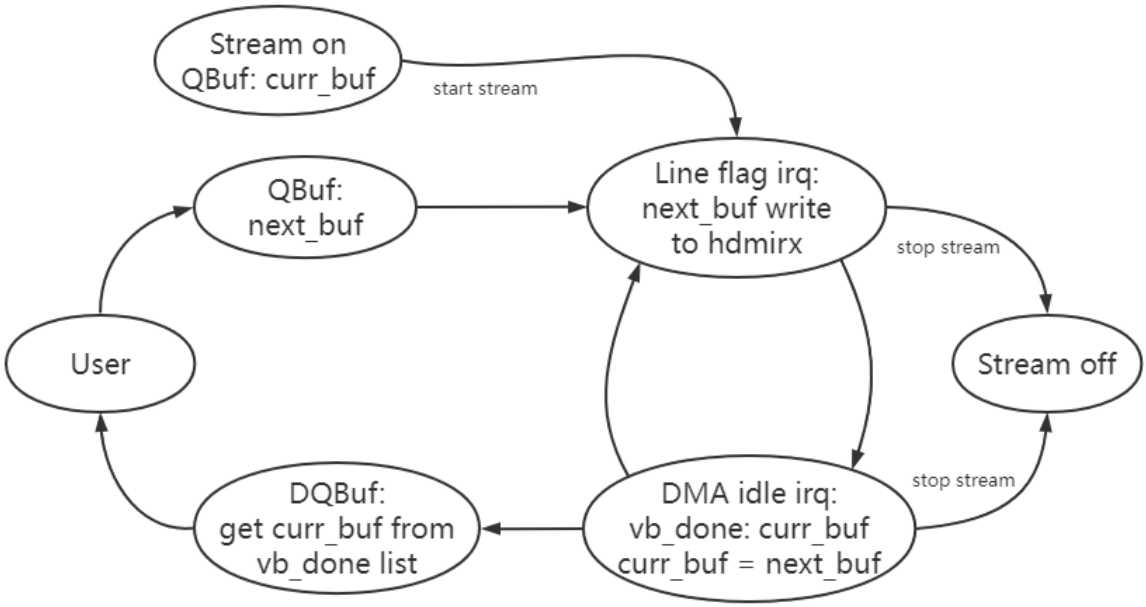
The resolution switching process is as follows:



The hot-plug procedure is as follows:



3.3 Image Buffer Rotation Mechanism



- QBuf/DQBuf: Users submit idle buffers via QBuf and retrieve buffers with completed image data filling via DQBuf.
- Line flag irq: Configures an interrupt to trigger at a fixed number of lines. Currently set to width/2 lines, generating an interrupt at the half-frame point. The buffer is updated within the interrupt handler. Writing the physical address of the buffer to the HDMI RX controller does not take effect immediately. It becomes effective only at the next vsync. Since the vblank duration is short, there may be insufficient time to update the buffer, potentially causing the next frame to overwrite the previous one. Therefore, the buffer must be updated in advance.
- DMA idle irq: DMA idle interrupt, which can be understood as a frame-end interrupt. This interrupt is triggered upon completion of a frame's image data transmission. Following the interrupt, the buffer is added to the `vb_done` list, awaiting retrieval by the user via `DQBuf`.
- Stream on: A command to start streaming. The initial buffer used is `curr_buf`.
- Stream off: A command to stop streaming. After the user issues this command, the driver waits for an interrupt before halting the data stream, and subsequently returns all buffers.

3.4 Image Transmission Latency

- After integrating with the TIF framework, display acceleration is achieved with a latency of approximately 20–30 ms.
- Integration with the Camera framework results in a display latency of approximately 100~120ms.

4. HDMI IN HDCP Function

Both HDCP1.4 and HDCP2.3 keys must be purchased by the customer from the official HDCP website. The RK3588 does not have built-in HDCP keys.

4.1 HDCP1.4

4.1.1 dts Configuration

Refer to the HDCP Configuration section and configure it as `hdcp1x-enable`.

4.1.2 Key Flashing

Key splitting and conversion tools are obtained from the SDK:

```
Under the `RKTools/windows/Rockchip_HdcpKey_Writer_V1.0.1.7z` toolkit, there is
a key splitting and conversion tool named `KeyConvertor`. Updating to
`KeyConvertor-V2.0.7` enables splitting such that each `.bin` file contains only
a single key.
```

The Key flashing tool is obtained from the SDK:

```
RKTools/windows/Rockchip_HdcpKey_Writer_V1.0.1.7z
```

- First, use the Key Splitting and Conversion Tool to extract a portion of the Key from the original Key file, and then convert this Key into a flashing Key with the `.skf` extension.
- If not using the flashing tool provided by Rockchip, the Key must be split into a file containing only a single Key. To achieve this, set the "Keys per File" parameter during original Key extraction to `1`, then click "Extract" to generate an individual Key file. Conversion to the `.skf` format is not required.
- Enter Loader mode on the device. In the flashing tool, select the "hdcp1.4Hdmirx" and "RK3588" options. The RK3588 HDCP1.4 Key incorporates AES encryption. Therefore, "Do AES" is an optional setting. If unchecked, only SEED encryption is applied. If checked, AES encryption is performed in addition to SEED encryption, providing enhanced security. Subsequently, import the aforementioned `.skf` file and click "Write" to complete the key flashing process.

Note: After the tool completes writing a Key, it automatically proceeds to the next Key.

- HDCP1.4 Key Structure Description.

```
An HDCP1.4 key consists of 308 bytes, detailed as follows:
8-Byte KSV: Composed of a 5-Byte KSV and 3 Bytes of 0x0.
280 Bytes DPK.
20 Bytes SHA.
```

4.1.3 HDCP1.4 Status View

```
cat /sys/class/misc/hdmirx_hdcp/status
HDCP Disable: HDCP1.4 is not enabled.
HDCP1.4: Authenticated start: During authentication
HDCP1.4: Authenticated success: Authentication successful
HDCP1.4: Authentication failed: Authentication failed
HDCP1.4: Unknown status: Unknown status
```

4.2 HDCP2.3

Current HDCP2.3 firmware packaging tool needs to be provided separately. Therefore, the HDCP2.3 toolkit must first be obtained via Redmine.

4.2.1 dts Configuration

Refer to the HDCP Configuration section and configure it as `hdcp2x-enable`.

4.2.2 Packaging Firmware and Boot Services

- Extract `WC_HDCP2_BASE_ESM_Firmware` to the Linux environment, then copy `hdcp_receivers.bin` to both the root directory and the `tools/` directory. If HDMITX support is also required, copy `hdcp_transmitter.bin` to the same directories simultaneously.

```
DWC_HDCP2_BASE_ESM_Firmware$
cp  ../hdcp_receivers.bin .
cp  ../hdcp_receivers.bin tools/
```

- Package the Firmware and RX KEY, and execute the **build_rockchip_fw.sh** script.

```
DWC_HDCP2_BASE_ESM_Firmware$ ./build_rockchip_fw.sh
```

The packaging process involves several steps that require user input:

1. Select the firmware packaging type: choose `1 - build firmware for both HDMIRX and HDMITX` to generate the **./firmware/hdcp2_hdmi.fw** firmware.

```
1 -build firmware for both HDMIRX and HDMITX
2 -build firmware for HDMITX only
3 -build firmware for DP
Choose the fw type: 1
```

2. The number of RX KEYs to be created shall be determined based on requirements (e.g., 10 or 100). Generate individual KEYs and store them in the `fw_hdcp_receivers_*.bin` file located in the `./rxkeys/hdcpkeys?-?/` directory. If the customer uses their own tool for flashing, the `fw_hdcp_receivers_*.bin` file can be flashed directly.

```
Create Rx Key Number:
10
......
Create 10 keys to tools/rxkeys/hdcpkeys1-10/
```

3. If the tools provided by Rockchip are required to flash the KEY, it must be packaged as a `.skf` file and stored in the `./rxkeys/hdcpkeys?-?/` directory. If Rockchip's tools are not required for flashing, select `n` here.

```
Do want to pack these keys to .skf file? [y/n]:
y

10 file packed to ./rxkeys/hdcpkeys1-10/hdcprxkeys1-10.skf
```

- HDCP2.3 RX KEY Flashing

1. Place `hdcp2_hdmi.fw` in the `device/rockchip/rk3588/` project directory. It will be copied to `vendor/firmware/hdcp2_hdmi.fw` during compilation.

2. Use the `RKDevInfoTool.exe` tool, select `HDCP2X_HDMIRX`, and import the previously generated `hdcprxkeys1-10.skf`. Enter Loader mode to perform flashing. After writing one Key, the tool will automatically proceed to the next Key and await flashing.
3. HDCP2.3 keys do not require AES encryption, therefore, only "RK3588" and "hdcp2x hdmirx" need to be selected.

- hdcp2_rx_tx service

The service loads automatically at boot. If an authentication exception occurs, use `logcat | grep HDCP2` to view the corresponding logs.

4.2.3 HDCP2.3 Status View

```
cat /sys/class/misc/hdmirx_hdcp/status

HDCP Disable: HDCP is not enabled
HDCP2.3: Authenticated success: Authentication successful
HDCP2.3: Authentication failed: Authentication failed
HDCP2.3: No decrypted: The source end has not enabled HDCP2.x.
```

4.3 HDCP KEY Flashing

1. Customers can use the flashing tools provided by Rockchip, which can be obtained from the `RKTools/windows/` directory in the SDK.
2. Customers can develop their own applications to flash keys. In this scenario, the `libhdcp.so` library provided by Rockchip can be utilized, allowing the application to directly invoke library interfaces for key encryption and subsequent flashing into vendor storage. This library is not included in the SDK by default and can be obtained by submitting a Redmine ticket.

The `libhdcp.so` library contains the following interfaces:

```
enum HDCP_KEY_ID {
    HDCP1X_KEY_HDMITX_RK33 = 0,
    HDCP1X_KEY_HDMITX_RK3588,
    HDCP1X_KEY_HDMIRX_RK3588,
    HDCP1X_KEY_HDMIRX_RK628,
    HDCP1X_KEY_DP_RK3588,
    HDCP2X_KEY_HDMIRX_RK3588,
};

/*
 * Encrypt the key, but is not written to vendor
 *
 * id: HDCP KEY type
 * keyin: HDCP1.x 308 Byte raw Key, HDCP2.x 1000 Byte Key
 * keyin_size: HDCP1.x 308, HDCP2.x 1000
 * keyout: The encrypted key
 * keyout_size: HDCP1.x 314, HDCP2.x 1000
 */
int hdcp_key_process(enum HDCP_KEY_ID id, uint8_t *keyin, int keyin_size,
uint8_t *keyout, int keyout_size);

/*
```

```
 * Encrypt key and write it to vendor
 *
 * id: HDCP KEY type
 * keyin: HDCP1.x 308 Byte raw Key, HDCP2.x 1000 Byte Key
 * keyin_size: HDCP1.x 308, HDCP2.x 1000
 */
int hdcp_key_process_and_write(enum HDCP_KEY_ID id, uint8_t *keyin, int
keyin_size);

/*
 * Write key to vendor
 *
 * id: HDCP KEY type
 * key: HDCP1.x 308 Byte raw Key, HDCP2.x 1000 Byte Key
 * size: key size
 */
int hdcp_key_write(enum HDCP_KEY_ID id, uint8_t *key, int size);

/*
 * Read key from vendor
 *
 * id: HDCP KEY type
 * key: HDCP1.x 308 Byte raw Key, HDCP2.x 1000 Byte Key
 * len: key length
 */
int hdcp_key_read(enum HDCP_KEY_ID id, uint8_t *key, int *len);
```

5. HDMI IN CEC Function

Modifications to `device/rockchip/common` are as follows:

TX: ro.hdmi.device_type=4 (Playback) ;

RX: ro.hdmi.device_type=0 (TV) ;

```
diff --git a/device.mk b/device.mk
index d600bf1..a5e9ae3 100644
--- a/device.mk
+++ b/device.mk
@@ -699,10 +699,10 @@ endif
 endif

 # hdmi cec
-ifneq ($(filter atv box, $(strip $(TARGET_BOARD_PLATFORM_PRODUCT))), )
+ifneq ($(filter atv box tablet, $(strip $(TARGET_BOARD_PLATFORM_PRODUCT))), )
 PRODUCT_COPY_FILES += \

frameworks/native/data/etc/android.hardware.hdmi.cec.xml:$(TARGET_COPY_OUT_VENDO
R)/etc/permissions/android.hardware.hdmi.cec.xml
-PRODUCT_PROPERTY_OVERRIDES += ro.hdmi.device_type=4
+PRODUCT_PROPERTY_OVERRIDES += ro.hdmi.device_type=0
 PRODUCT_PACKAGES += \
        hdmi_cec.$(TARGET_BOARD_PLATFORM)
```

```
+DEVICE_MANIFEST_FILE +=
device/rockchip/common/manifests/android.hardware.tv.cec@1.0-service.xml
 # HDMI CEC HAL
 PRODUCT_PACKAGES += \
diff --git a/manifests/android.hardware.tv.cec@1.0-service.xml
b/manifests/android.hardware.tv.cec@1.0-service.xml
new file mode 100644
index 00000000..5afa59d7
--- /dev/null
+++ b/manifests/android.hardware.tv.cec@1.0-service.xml
@@ -0,0 +1,11 @@
+<manifest version="1.0" type="device">
+    <hal format="hidl">
+        <name>android.hardware.tv.cec</name>
+        <transport>hwbinder</transport>
+        <version>1.0</version>
+        <interface>
+            <name>IHdmiCec</name>
+            <instance>default</instance>
+        </interface>
+    </hal>
+</manifest>
```

6. HDMI IN APK Adaptation Method

6.1 APK Source Code Path

- packages/apps/TV/partner_support/samples: Provides TV source data services, interacting with HAL layer, and preview APKs via the framework. As a hidden service that runs at boot, this APK has no icon on the home screen.
- packages/apps/rkCamera2: A preview APK that interacts with the aforementioned TV source data service via the framework layer. The icon name for this APK on the desktop is `HdmiIn`. Typically, customers will perform secondary development to replace it with their own preview APK.
- hardware/rockchip/tv_input: HAL layer code responsible for command interaction with the driver for stream switching, hot-plugging, and resolution change events.
- The HDMI IN feature is disabled by default in the SDK code. To enable the HDMI IN feature, configure the following properties. This will trigger the compilation of relevant modules, including the aforementioned APK.

```
vim device/rockchip/rk3588/BoardConfig.mk
BOARD_HDMI_IN_SUPPORT := true
```

After enabling `BOARD_HDMI_IN_SUPPORT`, the property `vendor.hwc.enable_sideband_stream_2_mode` is configured to `1` by default. Upon system boot, use `getprop` to verify whether this value has been set successfully.

6.2 HdmiIn Preview APK Instructions

- MainActivity main interface, utilizing the TIF (reusing part of the Android TV Input Framework) preview method, supports data preview for both HDMI-to-MIPI and HDMI RX paths. If developing a custom preview APK, the standard TvView control must be used. For driver-related configurations regarding HDMI-to-MIPI, refer to the document <Android 12+ Version HDMI IN Bridge Chip Developer Guide>.

```
String INPUT_ID =
"com.example.partnersupportsampletvinput/.SampleTvInputService/HW0";
Uri channelUri = TvContract.buildChannelUriForPassthroughInput(INPUT_ID);
// The value of `tvinput.hdmiin.type`: 0 for HDMI RX, 1 for HDMI TO MIPI. This
must be configured before tuning. If not set, it defaults to HDMI RX.
//SystemProperties.set("tvinput.hdmiin.type", "1");
tvView.tune(INPUT_ID, channelUri);
```

Note: If the product lacks HDMI RX and only supports HDMI to MIPI (e.g., RK3576, or RK3588 scenarios where HDMI RX is disabled in the DTS), the `tvinput.hdmiin.type` property must be configured in the system. Otherwise, the TIF solution will result in a black screen and fail to preview HDMI to MIPI. The configuration example for RK3576 is as follows:

```
device/rockchip/rk3576/device.mk
PRODUCT_PROPERTY_OVERRIDES += \
    tvinput.hdmiin.type=1
```

For the TIF preview method, the display submission approach requires enabling the `vtunnel` configuration and setting the `vendor.hwc.enable_sideband_stream_2_mode` property. The default DTS configuration for enabling `vtunnel` on the 3588 EVB1 development board is as follows, and the modifications should be made in the DTS for different device models:

```
diff --git a/arch/arm64/boot/dts/rockchip/rk3588-evb1-lp4.dtsi
b/arch/arm64/boot/dts/rockchip/rk3588-evb1-lp4.dtsi
index a10dad37f9cf..b3dd3a8bbb19 100644
--- a/arch/arm64/boot/dts/rockchip/rk3588-evb1-lp4.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3588-evb1-lp4.dtsi
@@ -275,6 +275,10 @@ &dp1_in_vp2 {
        status = "okay";
 };

+&rkvtunnel {
+        status = "okay";
+};
+
 /*
  * mipi_dcphy0 needs to be enabled
  * when dsi0 is enabled
diff --git a/arch/arm64/boot/dts/rockchip/rk3588.dtsi
b/arch/arm64/boot/dts/rockchip/rk3588.dtsi
index fa4a7e35099e..e3acf575aefd 100644
--- a/arch/arm64/boot/dts/rockchip/rk3588.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3588.dtsi
@@ -8,6 +8,10 @@
 #include "rk3588-vccio3-pinctrl.dtsi"

 / {
+        rkvtunnel: rkvtunnel {
+                compatible = "rockchip,video-tunnel";
+                status = "disabled";
```

```
+        };
         aliases {
                 dp0 = &dp0;
                 dp1 = &dp1;
diff --git a/arch/arm64/configs/rockchip_defconfig
b/arch/arm64/configs/rockchip_defconfig
index c742a4cc4fe5..039173d51bbe 100644
--- a/arch/arm64/configs/rockchip_defconfig
+++ b/arch/arm64/configs/rockchip_defconfig
@@ -684,6 +684,7 @@ CONFIG_ROCKCHIP_MPP_IEP2=y
 CONFIG_ROCKCHIP_MPP_JPGDEC=y
 CONFIG_ROCKCHIP_MPP_AV1DEC=y
 CONFIG_ROCKCHIP_MPP_VDPP=y
+CONFIG_ROCKCHIP_VIDEO_TUNNEL=y
 CONFIG_SOUND=y
 CONFIG_SND=y
 CONFIG_SND_DYNAMIC_MINORS=y
```

Note: For the TIF method, 5 buffers are required for rotation. To support BGR888, RGB888, and NV24 format data at 3840x2160 resolution, the theoretical CMA size is `3840 * 2160 * 3 * 5 = 121500KB ≈ 119M`. Considering that the system itself may occupy part of the CMA resources and fragmentation may occur, it is recommended to allocate a larger size, such as 256M. For instructions on how to increase the allocation, please refer to the section "2.3.2 Reserved Memory" above.

As shown in the example below, if logcat displays `cma: Out of memory` and `cat` reveals fewer than 5 blocks of allocated CMA memory, it indicates that the reserved CMA memory is insufficient.

```
rk3588_u:/ # cat /d/dma_buf/bufinfo | grep cma
24883200        00000002        00080007        00000002        cma     00000416
        cma
24883200        00000002        00080007        00000002        cma     00000415
        cma
24883200        00000002        00080007        00000002        cma     00000414
        cma
24883200        00000002        00080007        00000002        cma     00000413
        cma
rk3588_u:/ #

W tv_input_Sideband: dequeueBuffer 259 do allocateBuffer
I android.hardware.graphics.allocator-V1-service: alloc_format: FMT:0x113,MOD:0
I android.hardware.graphics.allocator-V1-service:
mali_gralloc_adjust_dimensions: alloc_format=FMT:0x113,MOD:0
usage=0x11000000a0000833 alloc_width=3840, alloc_height=2160
D cma     : __cma_alloc(cma 00000000a43238b2, count 6075, align 8)
E cma     : __cma_alloc: cma: alloc failed, req-size: 6075 pages, ret: -12
I cma     : number of available pages:
1@543+1@575+4@588+2@598+2@606+2@638+2@1694+2@1726+2@1758+2@1766+2@1774+2@1782+2@
1790+2@1798+2@1806+2@1814+2@1822+2@1830+2@1838+2@1846+2@1854+2@1862+2@1870+2@187
8+2@1886+2@1894+2@1902+2@1910+2@1918+2@1926+2@1934+2@1942+2@1950+2@1958+2@1966+2
@1974+2@1982+2@1990+2@1998+2@2006+2@2014+2@2022+2@2030+2@2038+2@2046+2@2054+2@20
62+2@2070+2@2078+2@2086+2@2094+2@2102+2@2110+2@2118+2@2126+2@2134+2@2142+2@2150+
2@2158+2@2166+2@2174+2@2182+114@2190+69@8379+69@14523+69@20667+5957@26811=> 6402
free of 32768 total pages
D cma     : __cma_alloc(): returned 0000000000000000
E DMABUFHEAPS: Unable to allocate from DMA-BUF heap: cma: Out of memory
E DMABUFHEAPS: No ion heap of name cma exists
E DMABUFHEAPS: No ion heap of name cma exists
```

```
E android.hardware.graphics.allocator-V1-service: libdmabufheap allocation
failed for cma heap
E android.hardware.graphics.allocator-V1-service: buffer allocation failed: Out
of memory
E android.hardware.graphics.allocator-V1-service: buffer allocation failed: Out
of memory
E GraphicBufferAllocator: Failed to allocate (3840 x 2160) layerCount 1 format
40 usage 11000000a0000833: 1
```

- RockchipCamera2 interface, Camera preview mode, supports HDMI to MIPI and HDMI RX path preview. By default, clicking the app icon launches the MainActivity interface. To enable camera path preview, first enable the HDMI IN camera function and configure the property:

```
vim device/rockchip/rk3588/BoardConfig.mk
CAMERA_SUPPORT_HDMI := true
```

Additionally, the property `persist.sys.hdmiinmode` must be configured to a value of 2. In this mode, launching the HdmiIn application initiates the RockchipCamera2 interface for data preview via the camera method. Alternatively, the system's built-in camera application can also be used for preview.

6.3 Differences Between TIF and Camera Preview Modes

	TIF	**Camera**
Advantages	Low latency	The app can obtain preview data for post-processing
Disadvantages	Does not support screen rotation, split-screen, picture-in-picture, or heterogeneous display features. The app cannot access preview buffer data. Does not support screenshot commands via the screencap method.	Higher latency than TIF

1. TIF preview does not support Picture-in-Picture (PIP) functionality. To utilize PIP, switch from the TIF scheme to the camera scheme. In the TIF preview sample `MainActivity`, tapping any location on the screen displays a dialog containing a "PIP" button, which facilitates switching from TIF preview to camera-based PIP preview. To observe this effect, ensure that `CAMERA_SUPPORT_HDMI := true` is set, indicating that the camera preview scheme is enabled.
2. Under TIF preview, screen recording and screenshot capture are supported via virtual display creation using the Google-standard `MediaProjectionManager`, as well as through the `screenrecord` command.

If screen recording and screenshot capabilities are required, or if PC-based screen mirroring via ADB is needed, the following property must be configured:

```
vim device/rockchip/rk3588/device.mk
PRODUCT_PROPERTY_OVERRIDES += debug.sf.enable_hwc_vds=true
```

If the product directory under `device` has been overridden, it must be configured within the corresponding product directory. This can be executed via `adb` after boot.

Run `getprop debug.sf.enable_hwc_vds` to verify whether the printed value is `true`, confirming the effectiveness of the modification.

3. Under TIF preview, the `adb screencap` command for screenshots is not supported, including shortcut button screenshots such as Volume Down + Power. Instead, the Google standard `MediaProjectionManager` virtual screen screenshot method mentioned in point 2 above is required.

7. Driver Debugging Methods

7.1 Debugging Tool Acquisition

Debugging requires the `v4l2-ctl` tool, which is automatically copied and integrated during SDK firmware compilation. It is specifically located in the SDK directory:

```
hardware/rockchip/camera/etc/tools/
```

7.2 Debugging Command Examples

When debugging and analyzing issues, it is recommended to enable debug logs. Refer to the section Enable the Log Switch.

7.2.1 View the video node of HDMIRX

```
grep -H hdmirx /sys/class/video4linux/video*/name
```

7.2.2 Locate the rk_hdmirx Device

Use the `v4l2-ctl -d` parameter to specify the video node, and the `-D` command to view node information. Confirm which node corresponds to the `rk_hdmirx` device via the Driver name:

```
rk3588_s:/ # v4l2-ctl -d /dev/video17 -D
Driver Info:
        Driver name      : rk_hdmirx
        Card type        : rk_hdmirx
        Bus info         : fdee0000.hdmirx-controller
        Driver version   : 5.10.66
        Capabilities     : 0x84201000
                Video Capture Multiplanar
                Streaming
                Extended Pix Format
                Device Capabilities
        Device Caps      : 0x04201000
                Video Capture Multiplanar
                Streaming
                Extended Pix Format
```

7.2.3 Retrieve Driver Timing Information

Retrieve the timings information saved by the device when the signal is locked:

```
rk3588_s:/ # v4l2-ctl -d /dev/video17 --get-dv-timings
DV timings:
        Active width: 3840
        Active height: 2160
        Total width: 4400
        Total height: 2250
        Frame format: progressive
        Polarities: -vsync -hsync
        Pixelclock: 594024000 Hz (60.00 frames per second)
        Horizontal frontporch: 172
        Horizontal sync: 92
        Horizontal backporch: 296
        Vertical frontporch: 8
        Vertical sync: 10
        Vertical backporch: 72
        Standards:
        Flags:
```

7.2.4 Real-time Query of Timing Information

Read timing information from the HDMI RX registers in real time:

```
rk3588_s:/ # v4l2-ctl -d /dev/video17 --query-dv-timings
        Active width: 3840
        Active height: 2160
        Total width: 4400
        Total height: 2250
        Frame format: progressive
        Polarities: -vsync -hsync
        Pixelclock: 594024000 Hz (60.00 frames per second)
        Horizontal frontporch: 172
        Horizontal sync: 92
        Horizontal backporch: 296
        Vertical frontporch: 8
        Vertical sync: 10
        Vertical backporch: 72
        Standards:
        Flags:
```

When executing `query-dv-timings`, if the debug level is configured to 2, viewing the kernel log via `dmesg` will display detailed timing information, including pixel format, color depth, TMDS clock, etc., as referenced below:

```
[16750.029542][ T2247] fdee0000.hdmirx-controller: hdmirx_get_pix_fmt: pix_fmt:
YUV422
[16750.029581][ T2247] fdee0000.hdmirx-controller: hdmirx_get_colordepth:
color_depth: 24, reg_val:4
[16750.029592][ T2247] fdee0000.hdmirx-controller: get timings from dma
[16750.029602][ T2247] fdee0000.hdmirx-controller: act:3840x2160,
total:4400x2250, fps:60, pixclk:594024000
[16750.029610][ T2247] fdee0000.hdmirx-controller: hfp:172, hs:92, hbp:296,
vfp:8, vs:10, vbp:72
[16750.029618][ T2247] fdee0000.hdmirx-controller: tmds_clk:594024000
[16750.029626][ T2247] fdee0000.hdmirx-controller: interlace:0, fmt:1, vic:127,
color:24, mode:hdmi
[16750.029633][ T2247] fdee0000.hdmirx-controller: deframer_st:0x11
[16750.029643][ T2247] fdee0000.hdmirx-controller: query_dv_timings:
3840x2160p60.00 (4400x2250)
```

7.2.5 Query Resolution and Image Format

Query the current resolution and image format:

```
rk3588_s:/ # v4l2-ctl -d /dev/video17 --get-fmt-video
Format Video Capture Multiplanar:
        Width/Height      : 3840/2160
        Pixel Format      : 'NV16'
        Field             : None
        Number of planes  : 1
        Flags             : premultiplied-alpha, 000000fe
        Colorspace        : Unknown (1025fcdc)
        Transfer Function : Unknown (00000020)
        YCbCr Encoding    : Unknown (000000ff)
        Quantization      : Default
        Plane 0           :
           Bytes per Line : 3840
           Size Image     : 16588800
```

7.2.6 Enable Image Data Stream

To enable the image data stream, configure the correct `video` node, resolution, and `pixelformat` according to actual requirements.

```
v4l2-ctl --verbose -d /dev/video17 \
--set-fmt-video=width=3840,height=2160,pixelformat='NV16' \
--stream-mmap=4
```

Note: RGB888 -> 'RGB3', YUV422 -> 'NV16', YUV420 -> 'NV12', YUV444 -> 'NV24'

7.2.7 Grabbing Image Files

Save the image file to the device. It can be pulled to the PC via `adb pull` and viewed using tools such as 7yuv or YUView.

```
v412-ctl --verbose -d /dev/video17 \
--set-fmt-video=width=3840,height=2160,pixelformat='NV16' \
--stream-mmap=4 --stream-skip=3 \
--stream-to=/data/4k60_nv16.yuv \
--stream-count=5 --stream-poll
```

7.2.8 Normal Stream Retrieval Log

If everything is working correctly and image data is received, the frame rate will be printed. Refer to the following log:

```
VIDIOC_QUERYCAP: ok
VIDIOC_G_FMT: ok
VIDIOC_S_FMT: ok
Format Video Capture Multiplanar:
        Width/Height      : 3840/2160
        Pixel Format      : 'NV16'
        Field             : None
        Number of planes  : 1
        Flags             : premultiplied-alpha, 000000fe
        Colorspace        : Unknown (1025fcdc)
        Transfer Function : Unknown (00000020)
        YCbCr Encoding    : Unknown (000000ff)
        Quantization      : Default
        Plane 0           :
           Bytes per Line : 3840
           Size Image     : 16588800
VIDIOC_REQBUFS: ok
VIDIOC_QUERYBUF: ok
VIDIOC_QUERYBUF: ok
VIDIOC_QBUF: ok
VIDIOC_QUERYBUF: ok
VIDIOC_QBUF: ok
VIDIOC_QUERYBUF: ok
VIDIOC_QBUF: ok
VIDIOC_QUERYBUF: ok
VIDIOC_QBUF: ok
VIDIOC_STREAMON: ok
idx: 0 seq:      0 bytesused: 16588800 ts: 103.172405
idx: 1 seq:      1 bytesused: 16588800 ts: 103.189072 delta: 16.667 ms
idx: 2 seq:      2 bytesused: 16588800 ts: 103.205738 delta: 16.666 ms
idx: 3 seq:      3 bytesused: 16588800 ts: 103.222404 delta: 16.666 ms
idx: 0 seq:      4 bytesused: 16588800 ts: 103.239070 delta: 16.666 ms fps:
60.00
idx: 1 seq:      5 bytesused: 16588800 ts: 103.255736 delta: 16.666 ms fps:
60.00
idx: 2 seq:      6 bytesused: 16588800 ts: 103.272402 delta: 16.666 ms fps:
60.00
```

8. Common Troubleshooting Methods

8.1 Enable the Log Switch

- The debug log level for the HDMI RX driver can be configured using the following command: 0-3.

```
echo 2 > /sys/module/rockchip_hdmirx/parameters/debug
dmesg -n 8
```

- To capture logs during the power-on boot process, it is recommended to directly modify the code and recompile and flash the kernel-related parts, as referenced in the following patch:

```
diff --git a/drivers/media/platform/rockchip/hdmirx/rk_hdmirx.c
b/drivers/media/platform/rockchip/hdmirx/rk_hdmirx.c
index c763a9558169..bd7f3effb45a 100644
--- a/drivers/media/platform/rockchip/hdmirx/rk_hdmirx.c
+++ b/drivers/media/platform/rockchip/hdmirx/rk_hdmirx.c
@@ -34,7 +34,7 @@
 #include "rk_hdmirx_cec.h"

 static struct class *hdmirx_class;
-static int debug;
+static int debug = 2;
 module_param(debug, int, 0644);
 MODULE_PARM_DESC(debug, "debug level (0-3)");

diff --git a/include/media/v4l2-common.h b/include/media/v4l2-common.h
index 1cc0c5ba16b3..e74f3a85f0b8 100644
--- a/include/media/v4l2-common.h
+++ b/include/media/v4l2-common.h
@@ -75,7 +75,7 @@
 #define v4l2_dbg(level, debug, dev, fmt, arg...)                        \
         do {                                                            \
                 if (debug >= (level))                                   \
-                        v4l2_printk(KERN_DEBUG, dev, fmt , ## arg);     \
+                        v4l2_printk(KERN_INFO, dev, fmt , ## arg);      \
         } while (0)
```

Note: It is generally not recommended to configure the debug level to 3, as it will generate a large volume of logs.

8.2 Read and Write Registers via io Commands

- The registers of the HDMI RX can be read and written via the `io` command. The following configuration must be enabled in the kernel config:

```
CONFIG_DEVMEM=y
```

- Example of querying registers using the io command:

```
// The following command can be used to view I/O usage statistics:
io -h
rk3588_s:/ # io -4 -l 0xc 0xfdee0594
fdee0594:  0000000f 80008000 00008000
```

8.3 HDMI RX Status Query

- Query the current status of the HDMI RX, including signal lock status, image format, Timings information, Pixel Clk, etc.:

```
rk3588_s:/ # cat /d/hdmirx/status
status: plugin
Clk-Ch:Lock     Ch0:Lock        Ch1:Lock        Ch2:Lock
Ch0-Err:0       Ch1-Err:0       Ch2-Err:0
Color Format: YUV422                    Store Format: YUV422 (8 bit)
Mode: 3840x2160p60 (4400x2250)          hfp:172  hs:92  hbp:296  vfp:8  vs:10
vbp:72
Pixel Clk: 594024000
```

- Query HDMI RX controller register information:

```
rk3588_s:/ # cat /d/hdmirx/ctrl

--------------------hdmirx ctrl--------------------
00000000: 48515258 30313130 6c753031 01000310
00000010: Reserved 31353138 30333039 32303231
00000020: WO...... 00211f01 198b7b25
00000040: WO...... 00001001 00000000
00000050: 00000001
...
---------------------------------------------------
```

- Query HDMI RX PHY register information:

```
rk3588_s:/ # cat /d/hdmirx/phy

--------------------hdmirx phy---------------------
0000004f: 01000000
0000100f: 01000000
0000110f: 01000000
0000120f: 01000000
0000130f: 01000000
0000104a: 01002600
...
---------------------------------------------------
```

- Query layer information:

```
rk3588_s:/ # cat /d/dri/0/summary
Video Port0: DISABLED
Video Port1: DISABLED
Video Port2: DISABLED
```

```
Video Port3: ACTIVE
Connector: DSI-1
bus_format[100a]: RGB888_1X24
overlay_mode[0] output_mode[0] color_space[0], eotf:0
Display mode: 1080x1920p60
clk[132000] real_clk[132000] type[48] flag[a]
H: 1080 1095 1099 1129
V: 1920 1935 1937 1952
Esmart3-win0: ACTIVE        //HDMI-IN preview layer
win_id: 11
format: RG24 little-endian (0x34324752) SDR[0] color_space[0] glb_alpha[0xff]
rotate: xmirror: 0 ymirror: 0 rotate_90: 0 rotate_270: 0
csc: y2r[0] r2y[0] csc mode[0]
zpos: 0
src: pos[0, 0] rect[1920 x 1080]
dst: pos[0, 0] rect[1080 x 1920]
buf[0]: addr: 0x00000000f070a000 pitch: 5760 offset: 0
Cluster3-win0: ACTIVE
win_id: 6
format: AB24 little-endian (0x34324241)[AFBC] SDR[0] color_space[0]
glb_alpha[0xff]
rotate: xmirror: 0 ymirror: 0 rotate_90: 0 rotate_270: 0
csc: y2r[0] r2y[0] csc mode[0]
zpos: 1
src: pos[0, 0] rect[1080 x 1920]
dst: pos[0, 0] rect[1080 x 1920]
buf[0]: addr: 0x00000000ef903000 pitch: 4352 offset: 0
```

8.4 HDMI IN Signal Lock Failure

The log for the HDMI IN signal lock failure is as follows:

```
[  285.949990][  T191] fdee0000.hdmirx-controller:
hdmirx_wait_lock_and_get_timing signal not lock, tmds_clk_ratio:0
```

The troubleshooting analysis steps are as follows:

- Measure the voltage level of the HDMI insertion detection pin `HDMIIRX_DET_L` to verify compliance with design specifications. Observe whether level transitions occur during HDMI connector insertion and removal.
- When HDMI is inserted, measure whether `HDMI_RX_HPD_PORT` at the HDMI port is pulled high normally. Verify whether it is pulled low upon removal.
- When HDMI is connected, use an oscilloscope to verify whether the TMDS signal is output correctly.
- Verify whether SCDC interaction is normal based on log prompts. According to the HDMI specification, when the pixel clock exceeds 340 MHz, `tmds_clk_ratio` must be configured to 1. Assuming the current source outputs a 4K60 image with a pixel clock of 594 MHz, but the log indicates `tmds_clk_ratio:0`, this signifies abnormal SCDC interaction. In such cases, inspect the HDMI DDC communication status or attempt re-plugging the connection.
- Query register status:

```
console:/ # io -4 0xfdee0050
fdee0150: 00000001 // bit0: 1 indicates HPD pulled high, 0 indicates HPD pulled
low

// 0x0594: bits[3:0] represent scdc_ch2locked, scdc_ch1locked, scdc_ch0locked,
and scdc_clockdetected;
// 0x0598: bit[31]: scdc_err_det1_valid, bit[15]: scdc_err_det0_valid; lower
bits represent error count statistics.
// 0x059c: bit[31]: scdc_erdet_lane0_valid, bit[15]: scdc_err_det2_valid; lower
bits represent error count statistics.
console:/ # io -4 -l 0xc 0xfdee0594
fdee0594: 0000000f 80008000 00008000 // Value during normal locking
```

- Some devices may probabilistically lock during plug/unplug operations. Attempt to extend the wait lock timeout to verify if locking can be completed:

```
diff --git a/drivers/media/platform/rockchip/hdmirx/rk_hdmirx.c
b/drivers/media/platform/rockchip/hdmirx/rk_hdmirx.c
index 39e4e15a6e17..a612fe30bda4 100644
--- a/drivers/media/platform/rockchip/hdmirx/rk_hdmirx.c
+++ b/drivers/media/platform/rockchip/hdmirx/rk_hdmirx.c
@@ -1264,7 +1264,7 @@ static int hdmirx_wait_lock_and_get_timing(struct
rk_hdmirx_dev *hdmirx_dev)
        u32 mu_status, scdc_status, dma_st10, cmu_st;
        struct v4l2_device *v4l2_dev = &hdmirx_dev->v4l2_dev;

-       for (i = 0; i < 300; i++) {
+       for (i = 0; i < 600; i++) {
                mu_status = hdmirx_readl(hdmirx_dev, MAINUNIT_STATUS);
                scdc_status = hdmirx_readl(hdmirx_dev, SCDC_REGBANK_STATUS3);
                dma_st10 = hdmirx_readl(hdmirx_dev, DMA_STATUS10);
@@ -1283,7 +1283,7 @@ static int hdmirx_wait_lock_and_get_timing(struct
rk_hdmirx_dev *hdmirx_dev)
                hdmirx_tmds_clk_ratio_config(hdmirx_dev);
        }

-       if (i == 300) {
+       if (i == 600) {
                v4l2_err(v4l2_dev, "%s signal not lock, tmds_clk_ratio:%d\n",
                                __func__, hdmirx_dev->tmds_clk_ratio);
                v4l2_err(v4l2_dev, "%s mu_st:%#x, scdc_st:%#x, dma_st10:%#x\n",
```

- Some signal generators or signal sources may exhibit poor signal quality. When connected to HDMI RX, this can result in high bit error rates (BER) on certain channels, leading to failure in locking. The presence of bit errors can be determined by reading registers 0xfdee0594-0xfdee059c:

```
rk3588_s:/ # io -4 -l 0xc 0xfdee0594
fdee0594:  0000000f 80008000 00008000       // Normal value

rk3588_s:/ # io -4 -l 0xc 0xfdee0594
fdee0594: 0000000f 804c8001 00008aff // Bit error detected, corresponding to the
total bit error counts for ch1, ch0, and ch2 respectively
```

If a channel fails to lock due to bit errors, the bit error detection function within the driver can be manually disabled to enhance compatibility:

```
diff --git a/drivers/media/platform/rockchip/hdmirx/rk_hdmirx.c
b/drivers/media/platform/rockchip/hdmirx/rk_hdmirx.c
index 606a08cd00505..6236196991c2a 100755
--- a/drivers/media/platform/rockchip/hdmirx/rk_hdmirx.c
+++ b/drivers/media/platform/rockchip/hdmirx/rk_hdmirx.c
@@ -1416,11 +1416,7 @@ static void hdmirx_controller_init(struct rk_hdmirx_dev
*hdmirx_dev)
                            CED_GBCHECKEN_QST |
                            CED_CTRLCHECKEN_QST |
                            CED_CHLOCKMAXER_QST_MASK,
-                           CED_VIDDATACHECKEN_QST|
-                           // CED_DATAISCHECKEN_QST |
-                           CED_GBCHECKEN_QST |
-                           CED_CTRLCHECKEN_QST |
-                           CED_CHLOCKMAXER_QST(0x10));
+                           0);
 }

 static void hdmirx_format_change(struct rk_hdmirx_dev *hdmirx_dev)
```

8.5 HDMI IN No Output/Black Screen Issue

The issue of no display output from HDMI IN can be discussed in two scenarios: driver-level anomalies or upper-layer and application functional anomalies. The basic troubleshooting approach is as follows:

Driver Troubleshooting:

1. Verify whether the kernel log is locked and if the resolution reading is normal. If a switch is utilized, also verify whether the switch output is properly locked. (Reference log: Hot-Swap Logs)
2. Determine whether each channel is locked and whether the resolution matches that of the source endpoint by checking the node information. (`cat /d/hdmirx/status`)
3. Determine whether the stream is normally enabled via logs or register status: `io -4 0xfdee4414`. if bit 1 is set, the stream is normally enabled.
4. Set the log level to 3 to check for the frame data. A log message is printed every half-frame.

Upper-layer troubleshooting:

1. Execute the `date` command to identify the approximate timestamp of the current issue.
2. cat /d/dri/0/summary Check the layer information.
3. dumpsys SurfaceFlinger.
4. Save the logcat logs.
5. After confirming the output stream and the presence of the layer from Step 2, capture a screenshot for verification. Use the following command:

```
android12:
rm -rf data/system/dumpimage
mkdir data/system/dumpimage
chmod 777 data/system/dumpimage
setprop vendor.hdmiin.debug.dump 1
setprop vendor.hdmiin.debug.dumpnum 5
Immediately after 1s
setprop vendor.hdmiin.debug.dumpnum 0
Immediately after 2 seconds
```

```
setprop vendor.hdmiin.debug.dump 0

android13:
rm -rf data/system/dumpimage
mkdir data/system/dumpimage
chmod 777 data/system/dumpimage
setprop vendor.tvinput.debug.dump 1
setprop vendor.tvinput.debug.dumpnum 10
Immediately after 0.5s
setprop vendor.tvinput.debug.dumpnum 0
Wait for 2s
setprop vendor.tvinput.debug.dump 0
```

Check if there are any files in `data/system/dumpimage`. If so, package the contents of `data/system/dumpimage` and provide them.

If no files are visible, it may be due to SELinux. Execute `setenforce 0` and re-run the aforementioned command.

6. Preserve the current state and avoid unnecessary operations. Provide the aforementioned information. If reproduction occurs and timely contact with us is impossible, making it infeasible to maintain the device state for an extended period, execute the following command to capture 10 seconds of data under such emergency conditions, provided that the aforementioned information has already been collected—specifically confirming that logcat data has been captured. (Note: This operation executes `logcat -c`, which clears previous logs. Do not perform this step unless necessary, and contacting us first is strongly recommended.)

```
setprop vendor.tvinput.debug.level 3
logcat -c
logcat
Capture for 10s
setprop vendor.tvinput.debug.level 0
```

Special Troubleshooting --- If the output stream and image capture are normal but the display is black, it may be caused by the `avmute` status. Investigate using the following command:

1. Check the `avmute` status bit:

```
io -4 0xfdee4458 //bit25 avmute status
io -4 0xfdee1030 //bit 0 GCP Set_AVMUTE bit4 GCP Clear_AVMUTE
```

2. Modify the color of avmute to check: (default: black)

```
io -4 0xfdee0430 0xff00 // Set RGB format avmute to red, and YUV format to other colors. If the color changes, it indicates that avmute is the cause.
```

Certain web pages or advertisement pages may employ HDCP encryption. If the HDCP function is not enabled on the sink side, it may result in AV mute.

8.6 Signal Instability and Loss of Lock Issue

During testing, some signal sources may exhibit instability, resulting in probabilistic black screens during preview or continuous log entries indicating repeated attempts to lock and acquire resolution, ultimately causing stream initialization failure. This issue is commonly observed with devices such as signal generators, TV boxes, and DVD players. Possible causes include: (refer to log: Loss of Lock After Signal Acquisition)

- The source end has HDCP encryption, but the sink end does not have HDCP functionality enabled, resulting in failure to lock.

Common HDCP-encrypted devices include: Apple computers, DVD players, and TV boxes.

- Poor cable quality, excessive cable length, loose connectors, etc., cause the HDMI signal input to the SoC to fail compliance with specifications.

Replace the cable and test for stability to determine if the issue is isolated to a specific cable.

- Excessive configuration of driver strength and swing in the front-end switch resulted in signal reflections and other impairments on the PCB traces.

Appropriately adjust the driver strength, swing, termination resistance, and other parameters of the front-end switch. Specific coordination with the switch OEM is recommended.

- Some interrupt configurations exhibit excessive sensitivity, causing disconnection and retry attempts upon detecting signal variations.

After closing the preview, determine which specific interrupt was triggered based on the logs, and verify whether the system operates normally after commenting out the relevant code. (It is recommended to submit a Redmine ticket for joint debugging.)

- The output signal from the source end is unstable during the initial period.

Add a delay to wait for the signal to stabilize before reporting the resolution.

```
  diff --git a/drivers/media/platform/rockchip/hdmirx/rk_hdmirx.c
b/drivers/media/platform/rockchip/hdmirx/rk_hdmirx.c
  index a69f8f6bd577..ede8d91f06dc 100644
  --- a/drivers/media/platform/rockchip/hdmirx/rk_hdmirx.c
  +++ b/drivers/media/platform/rockchip/hdmirx/rk_hdmirx.c
  @@ -1674,7 +1675,7 @@ static int hdmirx_wait_lock_and_get_timing(struct
rk_hdmirx_dev *hdmirx_dev)
           }

           hdmirx_reset_dma(hdmirx_dev);
  -        usleep_range(200*1000, 200*1010);
  +        usleep_range(500*1000, 500*1010);
           hdmirx_format_change(hdmirx_dev);

           return 0;
```

8.7 Source Resolution Switching Failed

Commonly found in Apple computers, laptops with NVIDIA GPUs, and devices running certain domestic operating systems. During resolution switching, the driver defaults to toggling the HPD signal low and then high to attempt reconnection, causing the source to interpret this as a sink disconnection and abort the operation. This can be resolved by configuring HPD as GPIO-controlled to maintain a constant high state throughout the resolution switching process.

Submit to Redmine to obtain the corresponding patch:
`rk3588_hdmirx_config_hpd_as_gpio_config_scdc_pwr_asap_230513.patch`

8.8 Driver Statistics TMDSCLK Errors

Commonly observed in 4k@60 RGB format input sources, where the source input TMDSCLK is 594M but statistically recorded as 148.5M, triggering errors in **frame rate** and **audio sampling rate**. Refer to the log: TMDSCLK Statistical Errors.

Submit to Redmine to obtain the corresponding patch: rk3588_a12_hdmirx_patch_fps_err_0714.patch

8.9 Source-End Special Scenario: Brief Black Screen

Commonly occurs when opening certain web pages (e.g., Youku, Bilibili) or resizing specific windows, leading to signal variations or increased bit error rates, which indirectly trigger interrupts or loss of lock.

1. Triggering an interrupt causes signal reconnection: By disabling the preview test, it is possible to observe from the log output which specific interrupt was triggered, and to verify whether disabling the corresponding interrupt resolves the issue. (It is recommended to submit a Redmine ticket.)
2. Increased bit errors cause loss of lock: Disable the bit error detection function. Refer to the section: HDMI IN Signal Lock Failure.

9. Typical Log Description

Generally, the debug level needs to be configured to 2 for relevant logs to be output via `dmesg`, or the code can be modified directly. Refer to the section: Enable the Log Switch.

9.1 Hot-Swap Logs

```
// Detects the unplug action
[29830.165185][ T2655] fdee0000.hdmirx-controller: hdmirx_delayed_work_hotplug:
plugin:0
// Disable interrupts
[29830.165203][ T2655] fdee0000.hdmirx-controller: hdmirx_interrupts_setup:
disable
// Pull HPD low
[29830.165211][ T2655] fdee0000.hdmirx-controller: hdmirx_hpd_ctrl: disable,
hpd_trigger_level:1
[29830.177109][  T598] fdee0000.hdmirx-controller: hdmirx_query_dv_timings port
has no link!
...
// Insertion action detected
[29843.128833][ T2655] fdee0000.hdmirx-controller: hdmirx_delayed_work_hotplug:
plugin:1
// Pull HPD high
[29843.139653][ T2655] fdee0000.hdmirx-controller: hdmirx_hpd_ctrl: enable,
hpd_trigger_level:1
```

```
...
[29843.247796][ T2655] fdee0000.hdmirx-controller: wait_reg_bit_status:  i:0,
time: 10ms
[29843.286353][ T2655] fdee0000.hdmirx-controller: wait_reg_bit_status:  i:38,
time: 50ms
[29843.286373][ T2655] rk_hdmirx fdee0000.hdmirx-controller:
hdmirx_audio_interrupts_setup: 1
// HDMI Signal Lock
[29843.703996][ T2655] fdee0000.hdmirx-controller:
hdmirx_wait_lock_and_get_timing signal lock ok, i:54!
...
// Image Format
[30098.978794][ T2655] fdee0000.hdmirx-controller: hdmirx_get_pix_fmt: pix_fmt:
YUV422
[30098.978803][ T2655] fdee0000.hdmirx-controller: hdmirx_get_colordepth:
color_depth: 24, reg_val:4
// Detailed resolution timing
[30098.978813][ T2655] fdee0000.hdmirx-controller: get timings from ctrl
[30098.978819][ T2655] fdee0000.hdmirx-controller: act:1920x1080,
total:2200x1125, fps:60, pixclk:148500000
[30098.978825][ T2655] fdee0000.hdmirx-controller: hfp:84, hs:48, hbp:148,
vfp:4, vs:5, vbp:36
// TMDS CLK
[30098.978829][ T2655] fdee0000.hdmirx-controller: tmds_clk:148500000
[30098.978835][ T2655] fdee0000.hdmirx-controller: interlace:0, fmt:1, vic:127,
color:24, mode:hdmi
[30098.978840][ T2655] fdee0000.hdmirx-controller: deframer_st:0x11
...
// Report resolution change event
[30099.023034][ T2655] fdee0000.hdmirx-controller: hdmirx_format_change: New
format: 1920x1080p60.00 (2200x1125)
[30099.023039][ T2655] fdee0000.hdmirx-controller: hdmirx_format_change: queue
res_chg_event
```

9.2 Resolution Switching Log

```
// Signal change, data error interrupt detected
[  312.662740][    C4] fdee0000.hdmirx-controller: avpunit_0_int_handler:
avp0_st:0x700000
[  312.662750][    C4] fdee0000.hdmirx-controller: mu2_st:0x2
// TMDS Signal Transition Interrupt
[  312.662756][    C4] fdee0000.hdmirx-controller: mainunit_2_int_handler:
TMDSVALID_STABLE_CHG
[  312.662760][    C4] fdee0000.hdmirx-controller: hdmirx_hdmi_irq_handler:
en_fiq
[  312.688916][  T196] fdee0000.hdmirx-controller: hdmirx_delayed_work_audio: no
supported fs(0), cur_state 0
[  312.688928][  T196] rk_hdmirx fdee0000.hdmirx-controller: audio off
// Enter the resolution switching process. The current HDMI status is plugged
in.
[  312.723309][  T196] fdee0000.hdmirx-controller:
hdmirx_delayed_work_res_change: plugin:1
[  312.723316][  T196] fdee0000.hdmirx-controller: hdmirx_interrupts_setup:
disable
[  312.724986][    C4] fdee0000.hdmirx-controller: mu0_st:0x4000000
```

```
[  312.724991][    C4] fdee0000.hdmirx-controller: hdmirx_hdmi_irq_handler:
en_fiq
// Pull HPD high after relevant configuration is completed
[  312.725000][  T196] fdee0000.hdmirx-controller: hdmirx_hpd_ctrl: enable,
hpd_trigger_level:1
...
// Signal Locking
fdee0000.hdmirx-controller: hdmirx_wait_lock_and_get_timing signal lock ok, i:2!
...
// Obtain the new resolution, image format, etc.
[  312.849040][  T196] fdee0000.hdmirx-controller: hdmirx_get_pix_fmt: pix_fmt:
YUV420
[  312.849049][  T196] fdee0000.hdmirx-controller: hdmirx_get_colordepth:
color_depth: 24, reg_val:4
[  312.849056][  T196] fdee0000.hdmirx-controller: get timings from ctrl
[  312.849060][  T196] fdee0000.hdmirx-controller: act:3840x2160,
total:4400x2250, fps:60, pixclk:296996000
[  312.849064][  T196] fdee0000.hdmirx-controller: hfp:84, hs:48, hbp:148,
vfp:8, vs:10, vbp:72
[  312.849067][  T196] fdee0000.hdmirx-controller: tmds_clk:296996000
[  312.849070][  T196] fdee0000.hdmirx-controller: interlace:0, fmt:3, vic:127,
color:24, mode:hdmi
```

9.3 Driver Switch Flow Log

```
// Start streaming to enable DMA and print 30 frames of data
[ 2360.096789][  T473] fdee0000.hdmirx-controller: hdmirx_start_streaming:
start_stream cur_buf y_addr:0x10700000, uv_addr:0x10700000
[ 2360.096813][  T473] fdee0000.hdmirx-controller: hdmirx_start_streaming:
enable dma
[ 2360.689582][  T205] fdee0000.hdmirx-controller: rcv frames
...
// Stop streaming
[ 2475.818301][  T473] fdee0000.hdmirx-controller: stream start stopping
[ 2475.818575][  T205] fdee0000.hdmirx-controller: hdmirx_dma_irq_handler: stop
stream!
[ 2475.818613][  T473] fdee0000.hdmirx-controller: stream stopping finished
```

9.4 HDMI IN Data Stream Printing

```
// To view data flow logs, set the log level to 3. One log entry is printed
every half-frame, as shown below:
[ 2778.207227][  T205] fdee0000.hdmirx-controller: dma_irq st1:0x80, st13:1080
[ 2778.215891][  T205] fdee0000.hdmirx-controller: dma_irq st1:0x100, st13:540
[ 2778.223889][  T205] fdee0000.hdmirx-controller: dma_irq st1:0x80, st13:1080
[ 2778.232559][  T205] fdee0000.hdmirx-controller: dma_irq st1:0x100, st13:540
[ 2778.240557][  T205] fdee0000.hdmirx-controller: dma_irq st1:0x80, st13:1080
[ 2778.249222][  T205] fdee0000.hdmirx-controller: dma_irq st1:0x100, st13:540
[ 2778.257226][  T205] fdee0000.hdmirx-controller: dma_irq st1:0x80, st13:1080
[ 2778.265887][  T205] fdee0000.hdmirx-controller: dma_irq st1:0x100, st13:540
[ 2778.273887][  T205] fdee0000.hdmirx-controller: dma_irq st1:0x80, st13:1080
[ 2778.282557][  T205] fdee0000.hdmirx-controller: dma_irq st1:0x100, st13:540
```

9.5 Signal Unlocked Exception Log

```
// Signal not locked
[  285.949990][  T191] fdee0000.hdmirx-controller:
hdmirx_wait_lock_and_get_timing signal not lock, tmds_clk_ratio:0
[  285.950011][  T191] fdee0000.hdmirx-controller:
hdmirx_wait_lock_and_get_timing mu_st:0x0, scdc_st:0x0, dma_st10:0x10
...
// Incorrect resolution detected
[  257.222739][  T193] fdee0000.hdmirx-controller: get timings from dma
[  257.222744][  T193] fdee0000.hdmirx-controller: act:39024x0, total:17732x1,
fps:8375, pixclk:148500000
[  257.222749][  T193] fdee0000.hdmirx-controller: hfp:4294904560, hs:184,
hbp:41260, vfp:1, vs:0, vbp:0
[  257.222753][  T193] fdee0000.hdmirx-controller: tmds_clk:148500000
[  257.222758][  T193] fdee0000.hdmirx-controller: interlace:0, fmt:0, vic:127,
color:24, mode:dvi
// Continuously read incorrect resolution
[  257.254994][  T193] fdee0000.hdmirx-controller: hdmirx_try_to_get_timings:
res not stable!
```

9.6 Loss of Lock After Signal Acquisition

```
// Enable stream
[140.628993][ T1525] fdee0000.hdmirx-controller: hdmirx_start_streaming:
start_stream cur_buf y_addr:0x10b00000, uv_addr:0x10b00000
[140.629009][ T1525] fdee0000.hdmirx-controller: hdmirx_start_streaming: enable
dma
[140.632067][  T198] fdee0000.hdmirx-controller: line_flag_int_handler: last
have no dma_idle_irq
[141.223426][  T198] fdee0000.hdmirx-controller: rcv frames
// Signal change detected after 3s preview
[144.955132][    C4] fdee0000.hdmirx-controller: mu0_st:0xc0000000
[144.955142][    C4] fdee0000.hdmirx-controller: mu2_st:0x2
[144.955173][    C4] fdee0000.hdmirx-controller: mainunit_2_int_handler:
TMDSVALID_STABLE_CHG
[144.955179][    C4] fdee0000.hdmirx-controller: hdmirx_hdmi_irq_handler: en_fiq
[144.958942][ T1525] fdee0000.hdmirx-controller: stream start stopping
...
// After locking, report TMDSVALID_STABLE_CHG, TMDSQPCLK_LOCKED_CHG,
TMDSQPCLK_OFF_CHG, and AVP1 interrupt.
[  176.865529][   T55] fdee0000.hdmirx-controller: hdmirx_format_change: New
format: 1920x1200p59.99 (2592x1242)
[  176.873290][   T55] fdee0000.hdmirx-controller: hdmirx_format_change: queue
res_chg_event
[  176.873311][   T55] fdee0000.hdmirx-controller: hdmirx_set_ddr_store_fmt:
pix_fmt: RGB888, DMA_CONFIG1:0x12000001
[  176.873317][   T55] fdee0000.hdmirx-controller: hdmirx_interrupts_setup:
enable
[  176.873355][    C4] fdee0000.hdmirx-controller: hdmirx_hdmi_irq_handler: hdmi
irq not handled!
[  176.873358][    C4] fdee0000.hdmirx-controller: avp0:0x0, avp1:0x0, mu0:0x0,
mu2:0x0, pk0:0x800, pk2:0x0, scdc:0x0
```

```
[  177.389362][    C4] fdee0000.hdmirx-controller: avpunit_0_int_handler:
avp0_st:0x200000
[  177.389378][    C4] fdee0000.hdmirx-controller: mu2_st:0x2
[  177.389397][    C4] fdee0000.hdmirx-controller: mainunit_2_int_handler:
TMDSVALID_STABLE_CHG
[  177.389403][    C4] fdee0000.hdmirx-controller: hdmirx_hdmi_irq_handler:
en_fiq
[  177.390164][    C4] fdee0000.hdmirx-controller: mu0_st:0x10
[  177.390185][    C4] fdee0000.hdmirx-controller: mainunit_0_int_handler:
TMDSQPCLK_LOCKED_CHG
[  177.390189][    C4] fdee0000.hdmirx-controller: hdmirx_hdmi_irq_handler:
en_fiq
[  177.392165][    C4] fdee0000.hdmirx-controller: mu0_st:0x10
[  177.392185][    C4] fdee0000.hdmirx-controller: mainunit_0_int_handler:
TMDSQPCLK_OFF_CHG
[  177.392190][    C4] fdee0000.hdmirx-controller: hdmirx_hdmi_irq_handler:
en_fiq
[  177.393391][    C4] fdee0000.hdmirx-controller: mu2_st:0x2
[  177.393410][    C4] fdee0000.hdmirx-controller: mainunit_2_int_handler:
TMDSVALID_STABLE_CHG
[  177.393415][    C4] fdee0000.hdmirx-controller: hdmirx_hdmi_irq_handler:
en_fiq
[  177.397433][    C4] fdee0000.hdmirx-controller: avpunit_1_int_handler:
avp1_st:0x40000000
[  177.397445][    C4] fdee0000.hdmirx-controller: hdmirx_hdmi_irq_handler:
en_fiq
[  177.397994][    C4] fdee0000.hdmirx-controller: avpunit_1_int_handler:
avp1_st:0x80000000
[  177.398002][    C4] fdee0000.hdmirx-controller: hdmirx_hdmi_irq_handler:
en_fiq
```

9.7 TMDSCLK Statistical Errors

```
// The source actually inputs 4K@60, with the actual TMDSCLK being 594 MHz.
[147.366197][  T192] fdee0000.hdmirx-controller: get timings from ctrl
[147.366201][  T192] fdee0000.hdmirx-controller: act:3840x2160, total:4400x2250,
fps:16, pixclk:153864000
[147.366206][  T192] fdee0000.hdmirx-controller: hfp:176, hs:88, hbp:296, vfp:8,
vs:10, vbp:72
[147.366210][  T192] fdee0000.hdmirx-controller: tmds_clk:153864000,
pix_clk:153864000
[147.366214][  T192] fdee0000.hdmirx-controller: interlace:0, fmt:0, vic:0,
color:24, mode:hdmi
...
// Incorrect audio sampling rate caused by tmdsclk statistics error
[152.555134][  T187] rk_hdmirx fdee0000.hdmirx-controller:
hdmirx_delayed_work_audio: audio underflow and overflow 0x3000000, with fs
invalid 12400
[152.555191][  T187] rk_hdmirx fdee0000.hdmirx-controller:
hdmirx_audio_fifo_init
[152.758428][  T192] rk_hdmirx fdee0000.hdmirx-controller:
hdmirx_delayed_work_audio: audio underflow and overflow 0x3000000, with fs
invalid 12400
[152.758458][  T192] rk_hdmirx fdee0000.hdmirx-controller:
hdmirx_audio_fifo_init
```

```
[152.965070][  T192] rk_hdmirx fdee0000.hdmirx-controller:
hdmirx_delayed_work_audio: audio underflow and overflow 0x3000000, with fs
invalid 12400
[152.965093][  T192] rk_hdmirx fdee0000.hdmirx-controller:
hdmirx_audio_fifo_init
```

9.8 Hot-plugging or Resolution Change Reboot

Null Pointer Exception and Restart After Removal

```
[  453.482730][  T819] fdee0000.hdmirx-controller: hdmirx_set_fmt: req(3840,
2160), out(3840, 2160), fmt:0x33524742
[  453.482745][  T819] fdee0000.hdmirx-controller: hdmirx_get_content_type:
Graphics
[  453.482752][  T819] fdee0000.hdmirx-controller: hdmirx_get_pix_fmt: pix_fmt:
RGB888
[  453.987489][  T819] fdee0000.hdmirx-controller: C-Plane 0 size: 24883200,
Total imagesize: 24883200
[  453.987524][  T819] fdee0000.hdmirx-controller: hdmirx_set_fmt: req(3840,
2160), out(3840, 2160), fmt:0x33524742
[  453.987539][  T819] fdee0000.hdmirx-controller: hdmirx_get_content_type:
Graphics
[  453.987545][  T819] fdee0000.hdmirx-controller: hdmirx_get_pix_fmt: pix_fmt:
RGB888
[  454.111292][    C3] hrtimer: interrupt took 15194002 ns
[  454.144120][ T3686] Unable to handle kernel paging request at virtual address
ffffffc0c684007d
DDR 73dffea49e typ 23/09/20-14:18:32,fwver: v1.14
LPDDR4X, 2112MHz
channel[0] BW=16 Col=10 Bk=8 CS0 Row=16 CS1 Row=16 CS=2 Die BW=16 Size=2048MB
channel[1] BW=16 Col=10 Bk=8 CS0 Row=16 CS1 Row=16 CS=2 Die BW=16 Size=2048MB
channel[2] BW=16 Col=10 Bk=8 CS0 Row=16 CS1 Row=16 CS=2 Die BW=16 Size=2048MB

[ 4088.362084][T31719] Unable to handle kernel paging request at virtual address
0000000200000077
[ 4088.364060][  T268] dw-hdcp fde70000.hdcp: hdcp reset
[ 4088.366597][T31719] Mem abort info:
[ 4088.366599][T31719]   ESR = 0x96000005
[ 4088.366601][T31719]   EC = 0x25: DABT (current EL), IL = 32 bits
[ 4088.366602][T31719]   SET = 0, FnV = 0
[ 4088.366604][T31719]   EA = 0, S1PTW = 0
[ 4088.366604][T31719] Data abort info:
[ 4088.366606][T31719]   ISV = 0, ISS = 0x00000005
[ 4088.366607][T31719]   CM = 0, WnR = 0
[ 4088.366609][T31719] user pgtable: 4k pages, 39-bit VAs, pgdp=000000010318c000
[ 4088.366614][T31719] [0000000200000077] pgd=0000000000000000
[ 4088.367094][  T580] Unable to handle kernel write to read-only memory at
virtual address ffffffc008105928
[ 4088.367373][T31719] , p4d=0000000000000000
[ 4088.367375][    C1] ------------[ cut here ]------------
[ 4088.367386][    C1] WARNING: CPU: 1 PID: 0 at kernel/workqueue.c:1660
queue_delayed_work_on+0xb4/0x114
```

Restart after reporting HDMI RX-related interrupts upon unplugging

```
[  417.412262][  T827] fdee0000.hdmirx-controller: C-Plane 0 size: 24883200,
Total imagesize: 24883200
[  417.412293][  T827] fdee0000.hdmirx-controller: hdmirx_set_fmt: req(3840,
2160), out(3840, 2160), fmt:0x33524742
[  417.412308][  T827] fdee0000.hdmirx-controller: hdmirx_get_content_type:
Graphics
[  417.412314][  T827] fdee0000.hdmirx-controller: hdmirx_get_pix_fmt: pix_fmt:
RGB888
[  417.666427][    C4] fdee0000.hdmirx-controller: mu0_st:0xc0000000
[  417.666443][    C4] fdee0000.hdmirx-controller: hdmirx_interrupts_setup:
disable
[  417.666468][    C4] fdee0000.hdmirx-controller: mainunit_2_int_handler:
TMDSVALID_STABLE_CHG
[  417.666581][ T3437] ------------[ cut here ]------------
DDR 73dffea49e typ 23/09/20-14:18:32,fwver: v1.14
LPDDR4X, 2112MHz
channel[0] BW=16 Col=10 Bk=8 CS0 Row=16 CS1 Row=16 CS=2 Die BW=16 Size=2048MB
channel[1] BW=16 Col=10 Bk=8 CS0 Row=16 CS1 Row=16 CS=2 Die BW=16 Size=2048MB
channel[2] BW=16 Col=10 Bk=8 CS0 Row=16 CS1 Row=16 CS=2 Die BW=16 Size=2048MB
```

Stream Restart After Insertion

```
[   31.300604][  T556] fdee0000.hdmirx-controller: hdmirx_get_color_range:
color_range: limit
[   31.300662][  T556] fdee0000.hdmirx-controller: hdmirx_get_color_space: video
standard: xvYCC601
[   31.316905][  T556] fdee0000.hdmirx-controller: hdmirx_start_streaming:
start_stream cur_buf y_addr:0x10900000, uv_addr:0x10900000
[   31.316924][  T556] fdee0000.hdmirx-controller: hdmirx_start_streaming:
enable dma
[  417.666581][ T3437] ------------[ cut here ]------------
DDR 73dffea49e typ 23/09/20-14:18:32,fwver: v1.14
LPDDR4X, 2112MHz
channel[0] BW=16 Col=10 Bk=8 CS0 Row=16 CS1 Row=16 CS=2 Die BW=16 Size=2048MB
channel[1] BW=16 Col=10 Bk=8 CS0 Row=16 CS1 Row=16 CS=2 Die BW=16 Size=2048MB
```

Black (green) screen after insertion, no image output even after re-plugging or changing the signal source.

In scenarios where the signal source exhibits HDMI RX anomalies without causing bus errors, DMA malfunction may prevent data transfer, resulting in a black screen. By setting the log level to 3, verify whether DMA-related interrupts are generated alternately as expected. The absence of the `dma_idle_irq` interrupt indicates a similar issue.

Normal log:

```
[ 2778.207227][  T205] fdee0000.hdmirx-controller: dma_irq st1:0x80, st13:1080
[ 2778.215891][  T205] fdee0000.hdmirx-controller: dma_irq st1:0x100, st13:540
[ 2778.223889][  T205] fdee0000.hdmirx-controller: dma_irq st1:0x80, st13:1080
[ 2778.232559][  T205] fdee0000.hdmirx-controller: dma_irq st1:0x100, st13:540
[ 2778.240557][  T205] fdee0000.hdmirx-controller: dma_irq st1:0x80, st13:1080
[ 2778.249222][  T205] fdee0000.hdmirx-controller: dma_irq st1:0x100, st13:540
```

Exception log:

```
[ 2778.215891][  T205] fdee0000.hdmirx-controller: dma_irq st1:0x100, st13:540
[ 2778.232559][  T205] fdee0000.hdmirx-controller: dma_irq st1:0x100, st13:540
[ 2778.249222][  T205] fdee0000.hdmirx-controller: dma_irq st1:0x100, st13:540
[ 2778.265887][  T205] fdee0000.hdmirx-controller: dma_irq st1:0x100, st13:540
[ 2778.282557][  T205] fdee0000.hdmirx-controller: dma_irq st1:0x100, st13:540
```

10. Patch Notes Regarding Restart Issues

Regarding the HDMI RX restart issues, updating the SDK can reduce the probability of occurrence. The submitted code version is as follows:

SDK version	kernel
android-12.0-mid-rkr14	5.10
android-13.0-mid-rkr6	5.10
android-14.0-mid-rkr1	6.1
linux-5.10-gen-rkr5	5.10

Note: NVR-sdk-v1.7 does not include the latest reboot issue patch and needs to be provided separately.

If the above versions are not updated, additional patches must be applied. Refer to: https://redmine.rock-chips.com/documents/130

kernel-5.10

- Method 1: Directly save your modifications, apply the corresponding patch using `git am`, and then manually incorporate your changes.
- Method 2: Replace all driver source code under the `../hdmirx/` path, then apply patches 0009, 0010, and 0022, followed by your own modifications.

rkbin

The version of `rk3588_bl31_v1.41.elf` must be at least v1.39 or later (i.e., `rk3588_bl31_v1.39.elf`).

Version confirmation reference: `vim rkbin/RKTRUST/RK3588TRUST.ini`

Typically, updating to version v1.42 by default resolves other DDR-related issues. Refer to the patch briefing: https://redmine.rock-chips.com/issues/441328

After updating BL31, the corresponding DDR_BIN version must also be updated accordingly.

After updating BL31 and flashing U-Boot, the success of the replacement can be verified via the boot log. For example, the build time corresponding to **v1.42** is:

```
INFO:    Preloader serial: 2
NOTICE:  BL31: v2.3():v2.3-639-g87bcc5dfe:derrick.huang
NOTICE:  BL31: Built : 18:06:16, Sep  9 2023
```

After applying the patch, verify the code consistency of the following critical functions:

Interrupt Configuration:

```
static void hdmirx_interrupts_setup(struct rk_hdmirx_dev *hdmirx_dev, bool en)
{
    struct v4l2_bt_timings *bt = &hdmirx_dev->timings.bt;

    v4l2_dbg(1, debug, &hdmirx_dev->v4l2_dev, "%s: %sable\n",
             __func__, en ? "en" : "dis");

    if (en && bt->pixelclock > 590000000 && hdmirx_dev->pix_fmt !=
HDMIRX_YUV420) {
        hdmirx_update_bits(hdmirx_dev, VMON_CONTROL,
                   VMON_IRQ_THR_MASK, VMON_IRQ_THR_MASK);
        hdmirx_update_bits(hdmirx_dev, VMON_CONTROL2,
                   VMON_IRQ_VERTICAL_MASK |
                   VMON_IRQ_HORIZONAL_MASK,
                   VMON_IRQ_VERTICAL_SEL(0x1f) |
                   VMON_IRQ_HORIZONAL_SEL(0x1f));
        hdmirx_update_bits(hdmirx_dev, DEFRAMER_CONFIG0,
                   VS_REMAPFILTER_EN_QST |
                   VS_FILTER_ORDER_QST_MASK |
                   HS_FILTER_ORDER_QST_MASK,
                   VS_REMAPFILTER_EN_QST |
                   VS_FILTER_ORDER_QST(0x3) |
                   HS_FILTER_ORDER_QST(0x0));
    } else {
        hdmirx_update_bits(hdmirx_dev, VMON_CONTROL,
                   VMON_IRQ_THR_MASK, 0);
        hdmirx_update_bits(hdmirx_dev, VMON_CONTROL2,
                   VMON_IRQ_VERTICAL_MASK |
                   VMON_IRQ_HORIZONAL_MASK,
                   VMON_IRQ_VERTICAL_SEL(0x1f) |
                   VMON_IRQ_HORIZONAL_SEL(0x0));
        hdmirx_update_bits(hdmirx_dev, DEFRAMER_CONFIG0,
                   HS_FILTER_ORDER_QST_MASK,
                   HS_FILTER_ORDER_QST(0x3));
    }
```

Stream start:

```
static int hdmirx_start_streaming(struct vb2_queue *queue, unsigned int count)
{
    ...
    touch_flag = (hdmirx_dev->bound_cpu << 1) | 0x1;
    sip_hdmirx_config(HDMIRX_AUTO_TOUCH_EN, 0, touch_flag, 100);
    ...
    sip_hdmirx_config(HDMIRX_INFO_NOTIFY, 0, DMA_CONFIG6, HDMIRX_DMA_EN);
    ...
}
```

Special Modifications

In certain scenarios, some customer tests indicate an increased probability of reboot when passing through a switch or an adapter chip (DP2HDMI), requiring intervention and analysis by the adapter chip vendor. It is currently known that Lontium's LT8641 (switch) and LT8711 (DP2HDMI) have optimizations for hot-plug scenarios on the RK3588. Consultation with the vendor is required to obtain the corresponding firmware for testing.